

Vietnam National University Ho Chi Minh City
University of Technology
Faculty of Computer Science and Engineering



**SPECIALIZED PROJECT
REPORT**

**Peer-to-Peer Electric Motorbike
Sharing Platform for Local
Communities**

Major: Computer Science

THESIS COMMITTEE:

SUPERVISOR: Assoc. Prof. Trương Tuấn Anh, PhD

REVIEWER: Assoc. Dương Huỳnh Anh Đức, PhD

—o0o—

STUDENT 1: Dương Hoàng Long 2211873

STUDENT 2: Hàng Nhựt Long 2211874

STUDENT 3: Âu Nguyễn Hùng Mạnh 2211991

Ho Chi Minh City, June 2025



PROTESTATION

Authors

Dương Hoàng Long

Hàng Nhật Long

Âu Nguyễn Hùng Mạnh



ACKNOWLEDGEMENTS

Authors

Dương Hoàng Long

Hàng Nhật Long

Âu Nguyễn Hùng Mạnh



Abstract



Contents



List of Figures



List of Tables

1 Giới thiệu

1.1 Động cơ của đề tài

Bối cảnh thực tiễn

Việt Nam hiện nay là quốc gia có mật độ xe máy cao nhất Đông Nam Á, chiếm hơn 85% phương tiện giao thông cá nhân. Chính phủ đang thúc đẩy chuyển đổi sang xe máy điện để giảm ô nhiễm không khí, khí thải CO₂ và tiếng ồn đô thị. Tuy nhiên, chi phí sở hữu xe điện (bao gồm mua xe, bảo dưỡng, sạc pin) vẫn còn là rào cản với nhiều người dùng, đặc biệt là sinh viên và cư dân ở các đô thị lớn.

Trong bối cảnh đó, với nhu cầu di chuyển cá nhân ngày càng cao trong khi chi phí sở hữu phương tiện cá nhân tăng mạnh, mô hình chia sẻ phương tiện theo hình thức P2P trở thành một hướng đi tiềm năng. Việc phát triển nền tảng chia sẻ xe cá nhân không chỉ giúp giảm gánh nặng tài chính cho người dùng mà còn góp phần giảm thiểu ô nhiễm môi trường và tối ưu hóa việc sử dụng hạ tầng giao thông.

Đồng thời, sự phát triển của công nghệ số và các nền tảng thương mại điện tử tạo điều kiện thuận lợi để hiện thực hóa mô hình này. Việc triển khai một hệ thống P2P vehicle sharing có khả năng kết nối an toàn, minh bạch và thuận tiện sẽ là một giải pháp thiết thực và mang ý nghĩa xã hội lớn.

Vấn đề tồn tại:

- Phương tiện cá nhân trong cộng đồng **ít được sử dụng** (nhàn rỗi vào phần lớn thời gian).
- Các nghiên cứu về mô hình *sharing economy* (Airbnb, Uber, xe đạp điện công cộng, car-sharing) đều xuất phát từ thực tế:
 - Tài sản cá nhân (xe, nhà, dụng cụ) thường chỉ sử dụng **10–20% thời gian trong ngày**, phần còn lại bị **under-utilized (dư thừa/nhàn rỗi)**.
 - Với bối cảnh đô thị Việt Nam, các phương tiện cá nhân nói chung cũng gặp tình trạng tương tự: chỉ phục vụ nhu cầu di chuyển ngắn hạn, còn lại đỗ tại khu dân cư, ký túc xá hoặc bãi giữ xe.
- Người dùng ngắn hạn (ví dụ sinh viên, khách thuê trọ) có **nhu cầu di chuyển linh hoạt** nhưng không muốn/không thể mua xe.
- Các hệ thống chia sẻ xe điện tập trung (như DatBike thử nghiệm, VinFast thuê pin) **tốn chi phí vận hành lớn**, khó mở rộng đến từng khu dân cư nhỏ.

Tính cấp thiết:

Việc xây dựng một nền tảng **P2P sharing** cho phép cư dân **cho thuê – thuê lại xe điện** trong cùng cộng đồng giúp:

- Tăng hiệu suất sử dụng xe điện hiện có.
- Giảm chi phí cho người thuê (pay-per-use).
- Góp phần **thúc đẩy quá trình chuyển đổi xanh**, giảm lượng xe xăng cá nhân.
- Tạo thêm thu nhập cho chủ xe → khuyến khích sở hữu xe điện.

1.2 Mục tiêu

Mục tiêu tổng quát:

Xây dựng một hệ thống phần mềm cho phép người dân trong cùng cộng đồng (chung cư, khu dân cư, ký túc xá, trường học...) có thể chia sẻ hoặc thuê xe máy điện một cách an toàn, minh bạch và tiện lợi.

Mục tiêu cụ thể:

- Xây dựng hệ thống nền tảng P2P chia sẻ phương tiện cá nhân, cho phép người dùng đăng ký, cho thuê hoặc đặt thuê phương tiện thông qua ứng dụng.
- Đảm bảo các yếu tố tin cậy và an toàn trong giao dịch thông qua cơ chế xác minh danh tính, quản lý hồ sơ và hệ thống đánh giá người dùng.
- Quản lý giao dịch và đề xuất phương tiện phù hợp dựa trên vị trí, thời gian, và nhu cầu sử dụng.

1.3 Đối tượng và phạm vi nghiên cứu của đề tài

Đối tượng nghiên cứu:

- Người dùng cá nhân có nhu cầu thuê hoặc cho thuê xe máy điện.
- Các loại xe máy điện cá nhân có thể chia sẻ trong phạm vi nội bộ cộng đồng.
- Công nghệ phần mềm cho chia sẻ phương tiện (P2P sharing), bao gồm định vị, thanh toán, và phân tích dữ liệu.

Phạm vi nghiên cứu:

- Triển khai thử nghiệm trong phạm vi nhỏ (ví dụ: một khu dân cư, ký túc xá hoặc trường đại học). Không gian: 2–3 cộng đồng trong cùng một đô thị; bán kính phân tích 3–5 km quanh mỗi cộng đồng.
- Hệ thống bao gồm ứng dụng di động (mobile app) và nền tảng quản trị web.
- Không đi sâu vào xây dựng phần cứng hoặc thiết bị IoT cho xe điện, mà tập trung vào phần mềm quản lý và chia sẻ phương tiện.
- Chức năng đưa vào đánh giá: đăng nhập/join cộng đồng; tìm kiếm theo vị trí; quản lý/duyet xe; đặt xe + thanh toán sandbox; chat/thông báo cơ bản;
- Giả định và hạn chế: mẫu vừa phải (≈ 30 –50 xe; 100–150 người dùng; 300–500 lượt đặt).

1.4 Ý nghĩa

1.4.1 Ý nghĩa thực tiễn

- Giảm chi phí đi lại, tăng hiệu suất sử dụng xe điện.
- Góp phần thúc đẩy văn hóa chia sẻ và phát triển kinh tế xanh.
- Mở ra mô hình kinh tế P2P trong lĩnh vực phương tiện tại Việt Nam.

1.4.2 Ý nghĩa khoa học

Minh họa ứng dụng công nghệ P2P sharing vào lĩnh vực phương tiện. Kết hợp machine learning và decentralized learning trong phân tích nhu cầu di chuyển.

1.5 Cấu trúc báo cáo

- Chương 1: Giới thiệu đề tài, mục tiêu, phạm vi và ý nghĩa.
- Chương 2: Trình bày các kiến thức và công nghệ nền tảng.
- Chương 3: Tổng hợp và phân tích các công trình liên quan.
- Chương 4: Mô tả hệ thống đề xuất.
- Chương 5: Phân tích và thiết kế chi tiết.
- Chương 6: Hiện thực và kiểm thử.



- Chương 7: Đánh giá hệ thống.
- Chương 8: Tổng kết và hướng phát triển.

2 Kiến thức và công nghệ nền tảng

2.1 P2P Marketplace

Peer-to-Peer (P2P) marketplace là nền tảng số kết nối trực tiếp giữa những người sở hữu tài sản hoặc cung cấp dịch vụ với những người có nhu cầu sử dụng, mà không yêu cầu nền tảng đó phải sở hữu tài sản hay dịch vụ đó. Nó tạo môi trường để cá nhân tương tác, giao dịch, thuê mướn, mua bán hoặc chia sẻ sản phẩm/dịch vụ dựa trên niềm tin, đánh giá và quy tắc chung.

Các đặc điểm cốt lõi

1. **Nguồn cung do người dùng tạo ra:** Người sở hữu tài sản (ví dụ: nhà, xe, thiết bị) đăng tải thông tin lên nền tảng để cho thuê hoặc bán.
2. **Giao dịch trực tiếp giữa các cá nhân:** Không qua trung gian sở hữu tài sản. Nền tảng chỉ đóng vai trò cơ sở hạ tầng hỗ trợ (kết nối, thanh toán, đánh giá).
3. **Cơ chế tin cậy (trust & reputation):** Bao gồm hệ thống đánh giá, phản hồi (ratings & reviews), hồ sơ người dùng và xác thực danh tính nhằm đảm bảo cả người cho thuê và người thuê đều đáng tin cậy.
4. **Tính an toàn và bảo mật:** Hệ thống thanh toán bảo mật, cơ chế giải quyết tranh chấp, tuân thủ quy định pháp lý và bảo vệ quyền riêng tư của người dùng.

2.2 Các hình thức phổ biến của P2P Marketplace

P2P marketplace có nhiều dạng, tùy theo loại tài sản hoặc dịch vụ được chia sẻ. Dưới đây là các hình thức phổ biến dựa theo tài liệu từ *Flipkart Commerce Cloud* [flipkartCCC2025]:

Table 1: Các hình thức phổ biến của P2P Marketplace

Hình thức	Mô tả	Ví dụ
Sharing Economy	Cá nhân chia sẻ tài sản hoặc cung cấp dịch vụ cá nhân mà người khác cần sử dụng, theo thời gian hoặc theo dịch vụ.	Uber, Airbnb, các dịch vụ vệ sinh, dịch vụ nhỏ lẻ
Crowdfunding	Người khởi xướng dự án kêu gọi nhiều người góp vốn để thực hiện một sản phẩm hoặc một dự án chung.	Kickstarter, Indiegogo
Service Marketplace	Người có kỹ năng cung cấp dịch vụ cho người cần – dạy học, coaching, thiết kế, sửa chữa, nghệ thuật ...	Udemy, Fiverr, các nền tảng freelancer

Lợi ích chung của các mô hình P2P

- **Giảm chi phí khởi nghiệp:** Vì nền tảng không cần sở hữu lượng hàng tồn (inventory) lớn.
- **Hiệu ứng mạng (network effect):** Càng nhiều người dùng ở cả hai phía cung và cầu thì giá trị nền tảng càng tăng.
- **Khả năng mở rộng nhanh:** Nếu mô hình thị trường được thiết kế tốt và tính tin cậy được xây dựng vững chắc.

Thách thức chung

- Xây dựng niềm tin và kiểm soát chất lượng giữa các cá nhân.
- Đáp ứng quy định pháp lý, bảo hiểm, và trách nhiệm khi xảy ra sự cố.
- Đạt được số lượng người dùng đủ lớn để thị trường hoạt động hiệu quả (*critical mass*).
- Thiết lập hệ thống thanh toán, xử lý tranh chấp và hỗ trợ người dùng.

2.3 Sharing Economy

Mô hình Sharing Economy (kinh tế chia sẻ) là một nhánh đặc thù trong hệ sinh thái P2P Marketplace, trong đó các cá nhân chia sẻ hoặc cho thuê lại tài sản, nguồn lực của mình thông qua một nền tảng trung gian số. Thay vì sở hữu riêng biệt, người dùng có thể tối ưu hóa giá trị sử dụng tài sản bằng cách chia sẻ quyền sử dụng tạm thời cho người khác.

Theo Flipkart Commerce Cloud [flipkartCCC2025], bản chất của kinh tế chia sẻ nằm ở việc chuyển từ quyền sở hữu (ownership) sang quyền sử dụng (access). Đây là bước chuyển lớn trong hành vi tiêu dùng hiện đại, nhấn mạnh việc sử dụng tài nguyên hiệu quả, giảm lãng phí và tạo ra lợi ích cho cả hai phía: người sở hữu và người sử dụng.

Đặc điểm nổi bật của mô hình Sharing Economy:

- Tối ưu hóa tài sản nhàn rỗi: Cho phép cá nhân khai thác tài sản chưa được sử dụng hết (ví dụ xe hơi, nhà ở, thiết bị).
- Tăng cường tính bền vững: Giảm nhu cầu sản xuất mới, từ đó giảm tiêu thụ năng lượng và phát thải.
- Cộng đồng người dùng dựa trên niềm tin: Các giao dịch dựa vào hệ thống đánh giá, xác thực và uy tín cá nhân.

- Hỗ trợ công nghệ nền: Sử dụng dữ liệu lớn (Big Data), định vị (GPS), thanh toán điện tử, và các API kết nối để đảm bảo trải nghiệm liền mạch.

Các lĩnh vực ứng dụng phổ biến:

- Vận chuyển: Uber, Grab, BlaBlaCar.
- Lưu trú: Airbnb, Luxstay.
- Tài chính cộng đồng: Lending Club, GoFundMe.
- Dịch vụ cá nhân: TaskRabbit, Fiverr.

3 Công trình liên quan

3.1 Các mô hình sharing economy

GrabBike / BeBike (dịch vụ gọi xe 2 bánh)

Điểm mạnh:

- Nền tảng lớn, UX/UI quen thuộc.
- Tích hợp thanh toán điện tử, bản đồ định vị chính xác.

Điểm yếu:

- Không phải P2P (không chia sẻ tài sản cá nhân, mà là dịch vụ vận tải).
- Chi phí vận hành cao (phần trăm chiết khấu).
- Người dùng không được tự do chọn xe, chỉ chọn chuyển.

Xanh SM (VinFast – dịch vụ taxi/xe điện)

Điểm mạnh:

- Hệ sinh thái đồng bộ (xe, trạm sạc, quản lý vận hành).
- Hình ảnh thương hiệu mạnh, tạo niềm tin.

Điểm yếu:

- Mô hình tập trung, cần đầu tư hạ tầng lớn.
- Không hướng tới cộng đồng nhỏ lẻ (ví dụ chung cư, ký túc xá).

Airbnb (mô hình P2P sharing trong lĩnh vực lưu trú)

Điểm mạnh:

- Thành công toàn cầu trong sharing economy.
- Cơ chế đánh giá 2 chiều → xây dựng trust.

Điểm yếu:

- Chưa áp dụng cho phương tiện ở Việt Nam.
- Phụ thuộc rất nhiều vào quản lý cộng đồng & chính sách địa phương.

Hệ thống xe đạp điện công cộng (Hà Nội, TP.HCM)

Điểm mạnh:

- Quản lý tập trung, dễ giám sát.
- Chi phí rẻ cho người dùng.

Điểm yếu:

- Hạn chế khu vực, không linh hoạt.
- Không tận dụng tài sản nhàn rỗi của cư dân.

Table 2: So sánh các mô hình tiêu biểu trong lĩnh vực chia sẻ và di chuyển

Mô hình	Loại hình	Tính P2P	Chi phí	Linh hoạt	Mở rộng
GrabBike / BeBike	Dịch vụ gọi xe 2 bánh	★☆☆☆☆	★★☆☆☆	★★☆☆☆	★★★★☆
Xanh SM (VinFast)	Taxi/xe điện tập trung	☆☆☆☆☆	★★★★★	★☆☆☆☆	★★☆☆☆
Airbnb	Chia sẻ lưu trú	★★★★★	★★☆☆☆	★★★★★	★★★★★
Xe đạp điện công cộng	Dịch vụ công cộng	★☆☆☆☆	★★☆☆☆	★☆☆☆☆	★★☆☆☆
Đề tài nghiên cứu	Chia sẻ xe máy điện (P2P)	★★★★☆	★☆☆☆☆	★★★★☆	★★★★☆

Kết luận: Chưa có hệ thống nào tại Việt Nam kết hợp **P2P model** cho xe máy điện. Đây chính là khoảng trống thị trường và cơ hội nghiên cứu/triển khai.

4 Hệ thống đề xuất

4.1 Mô tả hệ thống

Hệ thống cho phép người dân trong cùng cộng đồng có thể chia sẻ và thuê xe máy điện qua ứng dụng di động.

Xác định Stakeholders

Stakeholder	Vai trò	Quan tâm chính
Người thuê xe	Người dùng cuối	Giá, vị trí gần, xe uy tín
Chủ xe	Người sở hữu tài sản	Quản lý xe, bảo mật thông tin, uy tín
Quản trị viên hệ thống	Quản lý nền tảng	Phát hiện vi phạm, xử lý tranh chấp
Đối tác thanh toán	Ví điện tử/ngân hàng	Tính ổn định và bảo mật
Cộng đồng	Nền tảng triển khai	Giảm ô nhiễm, tăng tiện ích

Lợi ích

Đối tượng	Lợi ích cụ thể
Người thuê	Chi phí thấp hơn so với sở hữu và bảo trì xe điện. Không cần lo về sạc pin, bảo trì nếu nền tảng hỗ trợ. Tiếp cận dễ hơn và linh hoạt hơn (giờ thuê, địa điểm thuê).
Chủ xe	Tạo dòng thu nhập từ thời gian xe không sử dụng. Giảm chi phí chờ sử dụng và hao phí tài sản nhàn rỗi. Được cộng đồng tin tưởng nếu xây dựng reputation tốt.
Cộng đồng / môi trường	Giảm số lượng xe chạy/xăng, giảm khí thải, ô nhiễm. Sử dụng năng lượng sạch nếu xe điện được sạc từ nguồn sạch. thúc đẩy văn hóa chia sẻ, tiết kiệm tài nguyên.
Nền tảng / quản trị viên	Chi phí thấp hơn (không cần đầu tư mua xe, bảo trì số lượng lớn). Dễ mở rộng sang các khu vực hoặc cộng đồng khác. Khai thác dữ liệu để cải tiến dịch vụ (AI dự báo nhu cầu, bảo trì, tối ưu hoạt động).

Thách thức

Vấn đề	Cách ảnh hưởng & biện pháp giảm thiểu
Tin cậy & kiểm soát chất lượng	Người thuê lo xe hỏng, pin yếu, chủ xe có thể bỏ bê bảo trì. Cần hệ thống đánh giá, feedback, kiểm định định kỳ, hình ảnh trạng thái xe.
Thanh toán & rủi ro tài chính	Cần đảm bảo thanh toán an toàn, xử lý hỏng tài sản, trộm cắp, va chạm. Giải pháp: có hợp đồng, bảo hiểm, khoản đặt cọc.
Quy định pháp lý / bảo hiểm	Luật về phương tiện giao thông đường bộ, đăng ký xe, trách nhiệm dân sự. Có thể cần giấy phép, bảo hiểm bắt buộc.
Đạt số lượng (critical mass)	Nếu không đủ chủ xe và người thuê thì dịch vụ không hiệu quả (xe ít, giá cao hoặc người thuê ít). Cần chiến lược thu hút sớm – ưu đãi, tiếp thị trong cộng đồng.
Chi phí hạ tầng & vận hành	Dù không cần sở hữu xe, nền tảng vẫn cần chi phí cho backend, ứng dụng, bản đồ/GPS, IoT đo trạng thái, support khách hàng.
Vấn đề vị trí & logistics	Xe phải được đỗ ở chỗ dễ tiếp cận. Di chuyển giữa các khu quan trọng. Quản lý pin và sạc điện. Nếu xe bị đặt ở nơi bất tiện, trải nghiệm người dùng bị ảnh hưởng.

4.2 Yêu cầu chức năng

Quản lý người dùng

- Đăng ký/đăng nhập tài khoản (email, OTP, OAuth).
- Phân quyền: người dùng cuối, chủ xe, quản trị viên.
- Quản lý thông tin cá nhân và phương thức thanh toán.
- Hệ thống đánh giá và xếp hạng người dùng.

Quản lý xe

- Đăng ký xe để chia sẻ (thông tin xe, hình ảnh, giá thuê).
- Cập nhật trạng thái xe (khả dụng, đang cho thuê, bảo trì).

- Theo dõi vị trí xe và tình trạng pin.
- Báo cáo sự cố hoặc hư hỏng.

Tìm kiếm và đặt xe

- Hiển thị xe trên bản đồ theo vị trí người dùng.
- Bộ lọc tìm kiếm (khoảng cách, giá cả, đánh giá).
- Đặt xe theo thời gian/quãng đường.
- Xác nhận đặt xe và thanh toán.

Theo dõi chuyến đi

- Bắt đầu/kết thúc chuyến đi.
- Theo dõi thời gian và quãng đường.
- Hiển thị chi phí thời gian thực.
- Báo cáo sự cố trong quá trình sử dụng.

Thanh toán

- Nhiều phương thức thanh toán (thẻ, ví điện tử).
- Tính phí theo thời gian/quãng đường.
- Xử lý tiền đặt cọc và hoàn tiền.
- Phân chia doanh thu giữa chủ xe và nền tảng.

Báo cáo và phân tích

- Thống kê sử dụng cho chủ xe.
- Báo cáo thu nhập và lịch sử thuê.
- Hệ thống phân tích dữ liệu sử dụng decentralized learning.
- Dự báo nhu cầu thuê xe theo khu vực và thời gian.

4.3 Yêu cầu phi chức năng

Hiệu suất

- Thời gian phản hồi nhanh (< 2 giây) cho các thao tác cơ bản.
- Hỗ trợ nhiều người dùng đồng thời trong một khu vực.
- Tối ưu hóa sử dụng dữ liệu di động.
- Xử lý được ít nhất 1000 giao dịch/giờ.

Khả năng mở rộng

- Kiến trúc cho phép mở rộng theo quy mô người dùng.
- Hỗ trợ thêm loại phương tiện mới trong tương lai.
- Dễ dàng thêm khu vực hoạt động mới.
- Tích hợp được với các dịch vụ bên thứ ba.

Bảo mật

- Mã hóa dữ liệu người dùng và thông tin thanh toán.
- Xác thực hai lớp cho các giao dịch quan trọng.
- Bảo vệ dữ liệu vị trí và lịch sử di chuyển.
- Tuân thủ quy định bảo vệ dữ liệu cá nhân.

Độ tin cậy

- Thời gian hoạt động $> 99.9\%$.
- Sao lưu dữ liệu tự động và phục hồi sau sự cố.
- Hoạt động ổn định trong điều kiện mạng không ổn định.
- Cơ chế quản lý lỗi và báo cáo sự cố.

Trải nghiệm người dùng

- Giao diện trực quan, dễ sử dụng.
- Tương thích với nhiều thiết bị (điện thoại, máy tính bảng).
- Hỗ trợ đa ngôn ngữ (tiếng Việt, tiếng Anh).
- Chế độ tiết kiệm pin và dữ liệu.



4.4 Data requirements

5 Phân tích hệ thống

5.1 Mô hình hóa tương tác

5.1.1 Use case diagram

Application Use Case

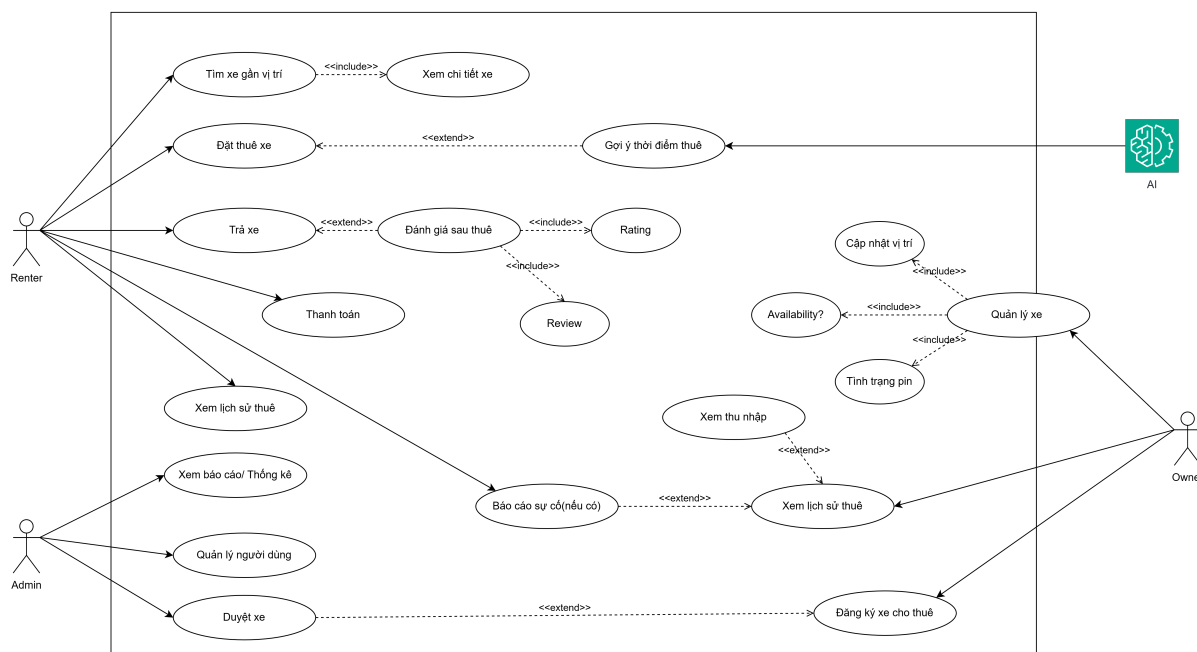


Figure 1: Application Use Case Diagram

- **UC-01:** Đăng ký / Đăng nhập
- **UC-13:** Đánh giá & Xếp hạng người dùng (Trust Score System)
- **UC-14:** Gợi ý thời điểm thuê (Recommendation)
- **UC-15:** Xem lịch sử thuê / thu nhập (của Owner hoặc Renter)
- **UC-16:** Báo cáo sự cố (Renter/Owner)
- **UC-17:** Thông báo
- **Renter Use Case**
- **Owner Use Case**
- **Admin Use Case**

Renter Use Case

- **UC-02:** Tìm xe gần vị trí
- **UC-03:** Xem chi tiết xe
- **UC-04:** Đặt thuê xe
- **UC-05:** Trả xe
- **UC-06:** Thanh toán
- **UC-07:** Đánh giá sau thuê (Rating / Review)

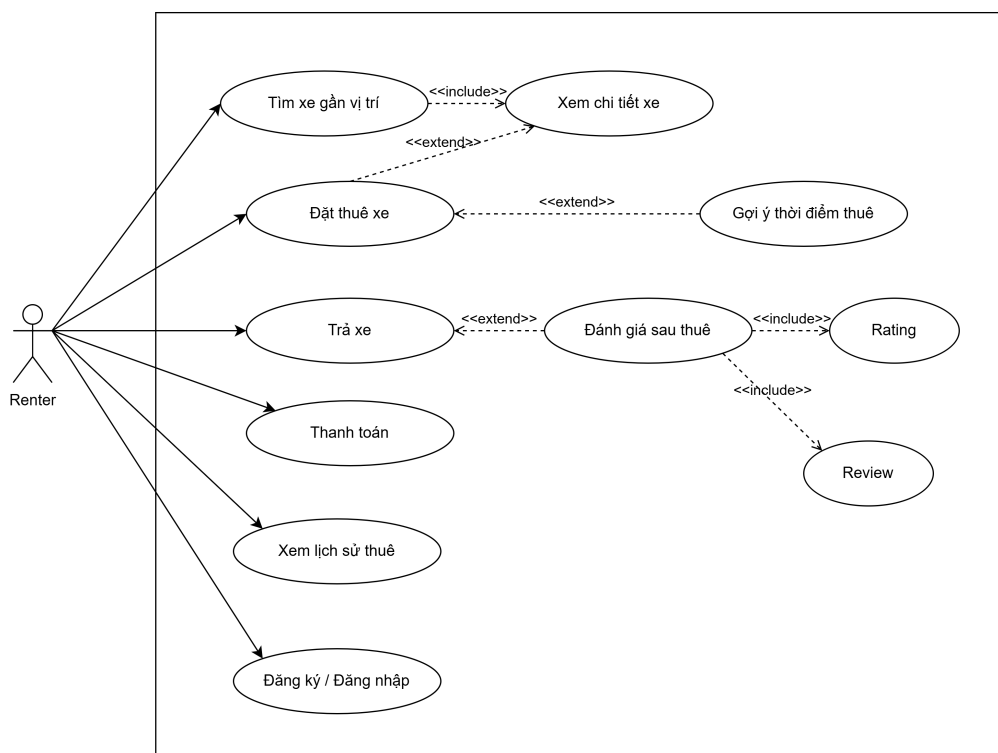


Figure 2: *Renter Use Case Diagram*

Owner Use Case

- **UC-08:** Đăng ký xe cho thuê (Owner)
- **UC-09:** Quản lý xe (Owner quản lý availability, cập nhật vị trí, pin)

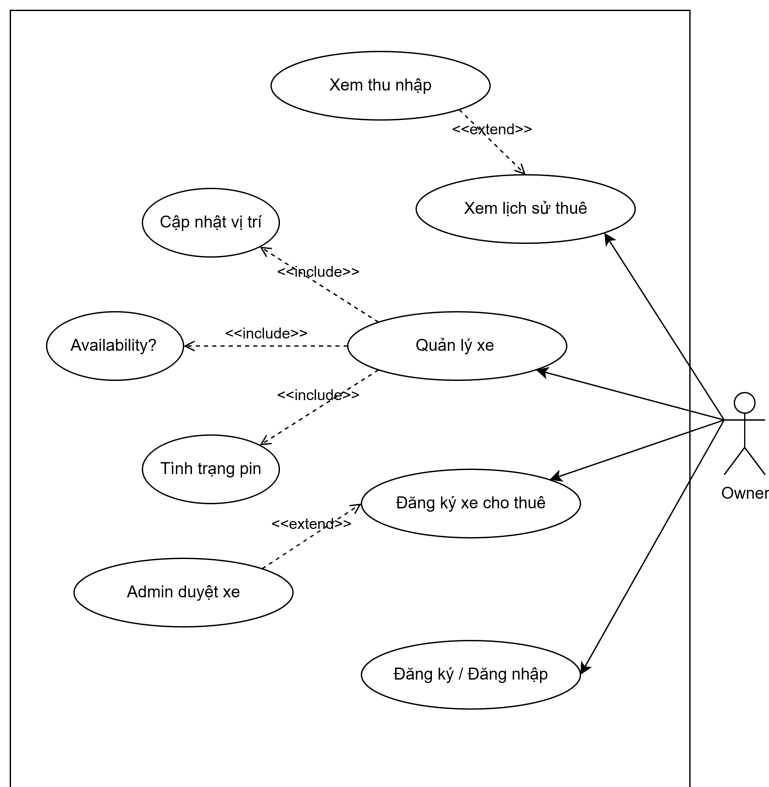


Figure 3: Owner Use Case Diagram

Admin Use Case

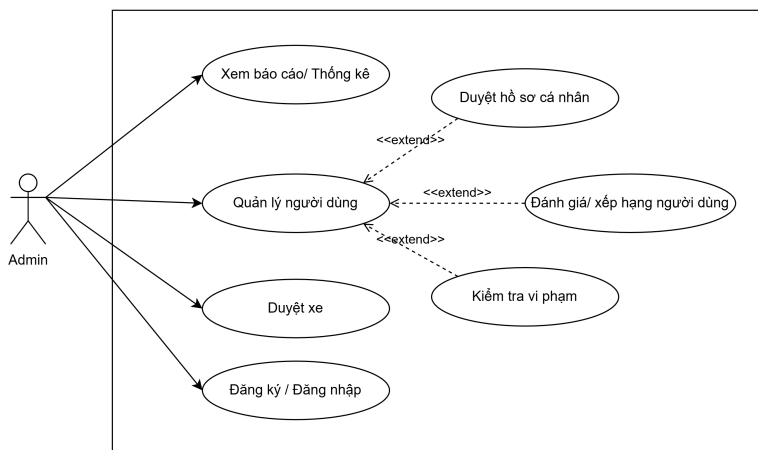


Figure 4: Admin Use Case Diagram

- **UC-10:** Duyệt xe / Quản trị hệ thống (Admin)
- **UC-11:** Xem báo cáo / Thống kê (Admin)
- **UC-12:** Quản lý người dùng

5.1.2 Use case description

UC-01 –Đăng ký / Đăng nhập

Trường	Mô tả
Actor	Người dùng (Customer / Vehicle Owner / Admin)
Description	Người dùng đăng ký tài khoản mới hoặc đăng nhập vào hệ thống.
Trigger	Người dùng chọn "Đăng ký / Đăng nhập" trên giao diện.
Precondition	Ứng dụng hoạt động, có kết nối mạng.
Postcondition	Người dùng được xác thực và truy cập vào giao diện cá nhân.
Normal Flow	1. Người dùng nhập email/số điện thoại và mật khẩu. 2. Hệ thống kiểm tra thông tin. 3. Nếu hợp lệ, đăng nhập thành công. 4. Chuyển hướng tới màn hình chính.
Alt Flow	A1. Người dùng chọn "Quên mật khẩu" → Hệ thống gửi OTP khôi phục. A2. Người dùng chọn "Đăng ký" → điền thông tin, gửi yêu cầu xác thực.
Exception Flow	E1. Sai mật khẩu quá 5 lần → tạm khóa tài khoản. E2. Hệ thống lỗi kết nối → hiển thị thông báo "Không thể đăng nhập, vui lòng thử lại."

UC-02 – Tìm xe gần vị trí

Trường	Mô tả
Tên Use Case	Tìm xe gần vị trí
Actor chính	Người thuê xe (Renter)
Mục tiêu	Giúp người dùng tìm xe khả dụng trong khu vực xung quanh, dựa trên vị trí hiện tại (GPS) hoặc vị trí nhập tay để đặt trước.

Trường	Mô tả
Trigger	Người dùng nhấn “Tìm xe gần tôi” hoặc chọn “Nhập vị trí khác”.
Precondition	Người dùng đã đăng nhập; có quyền truy cập bản đồ; GPS được bật hoặc người dùng nhập vị trí thủ công.
Postcondition	Danh sách xe khả dụng được hiển thị theo khu vực được chọn.
Normal Flow	1. Người dùng chọn “Tìm xe gần tôi”. 2. Hệ thống yêu cầu quyền truy cập vị trí GPS. 3. Ứng dụng lấy vị trí hiện tại của người dùng. 4. Backend truy vấn danh sách xe khả dụng trong bán kính. 5. Hệ thống hiển thị các xe gần nhất cùng thông tin: khoảng cách, mức pin, giá thuê, đánh giá. 6. Người dùng chọn một xe để xem chi tiết.
Alt Flow	1. Người dùng chọn “Nhập vị trí khác”. 2. Hệ thống hiển thị hộp tìm kiếm địa điểm. 3. Người dùng nhập địa chỉ hoặc khu vực mong muốn. 4. Hệ thống sử dụng Geocoding API để lấy tọa độ (lat, lng). 5. Backend truy vấn danh sách xe khả dụng quanh vị trí đó. 5.1. Nếu có xe khả dụng, hiển thị danh sách. 5.2. Nếu không có xe, gợi ý mở rộng bán kính hoặc chọn thời gian khác.
Exception Flow	E1. Không có xe khả dụng → hiển thị thông báo “Không có xe trong khu vực này.” E2. GPS bị tắt và người dùng không nhập vị trí → yêu cầu bật định vị hoặc nhập vị trí thủ công. E3. Lỗi kết nối mạng → hiển thị thông báo “Không thể tải dữ liệu. Vui lòng thử lại.”
Included Use Cases	UC-03 - Xem chi tiết xe
Extended Use Cases	UC-04 - Đặt thuê xe (Pre-booking)

Trường	Mô tả
Business Rules	- Vị trí nhập tay phải nằm trong khu vực hoạt động của hệ thống. - Với khoảng cách > 10km, hệ thống gợi ý chuyển sang khu vực gần hơn hoặc chế độ giao xe (nếu có). - Khi “Đặt trước”, chỉ hiển thị xe có trạng thái available trong khung thời gian đã chọn.

UC-03 – Xem chi tiết xe

Trường	Mô tả
Actor	Người thuê (Renter), Chủ xe (Owner)
Mục tiêu	Cho phép người dùng xem thông tin chi tiết của xe, bao gồm tình trạng pin, vị trí, giá thuê, đánh giá và thông tin chủ xe.
Trigger	Người dùng chọn một xe từ danh sách hiển thị (từ UC-02: "Tìm xe gần vị trí").
Precondition	Xe tồn tại và đang ở trạng thái "có sẵn (available)".
Postcondition	Thông tin chi tiết hiển thị cho người dùng.
Normal Flow	1. Người thuê chọn xe từ bản đồ hoặc danh sách. 2. Server trả thông tin: hình ảnh, loại xe, tình trạng pin, vị trí hiện tại, giá, đánh giá trung bình, chủ xe. 3. Ứng dụng hiển thị giao diện chi tiết xe. 4. Người thuê có thể chọn "Đặt thuê xe"
Alternative Flow	A1. Người thuê thêm xe vào danh sách "Yêu thích" (wishlist). A2. Người thuê chia sẻ đường dẫn xe cho người khác (social sharing).
Exception Flow	E1. Xe đã bị ẩn hoặc đang thuê → hiển thị thông báo "Xe không khả dụng". E2. Lỗi tải dữ liệu → hiển thị "Không thể tải thông tin xe".
Included Use Cases	Xem đánh giá (Review)

Trường	Mô tả
Extended Use Cases	Đặt thuê xe (UC-04)

UC-04 – Đặt thuê xe

Trường	Mô tả
Actor	Người thuê xe
Description	Đặt trước xe để thuê trong khoảng thời gian mong muốn.
Trigger	Người dùng nhấn "Đặt thuê" trên chi tiết xe.
Precondition	Xe đang ở trạng thái "Available", người dùng đã đăng nhập.
Postcondition	Đơn đặt xe được ghi nhận và chờ xác nhận.
Normal Flow	1. Người dùng chọn thời gian thuê. 2. Hệ thống gợi ý thời điểm thuê tối ưu (AI). 3. Người dùng xác nhận đặt thuê. 4. Hệ thống tạo đơn và gửi thông báo cho chủ xe.
Alt Flow	A1. Người thuê thay đổi thời gian thuê → hệ thống cập nhật chi phí tương ứng. A2. Chủ xe từ chối yêu cầu thuê.
Exception Flow	E1. Xe đã được người khác đặt trước → hiển thị thông báo lỗi. E2. Lỗi thanh toán hoặc xác nhận mạng.
Extended Use Cases	Gợi ý thời điểm thuê

UC-05 – Trả xe

Trường	Mô tả
Actor	Người thuê xe
Description	Người thuê kết thúc hành trình và trả xe.
Trigger	Người thuê chọn "Trả xe" trên giao diện.

Trường	Mô tả
Precondition	Xe đang trong trạng thái thuê.
Postcondition	Xe được cập nhật trạng thái "Available".
Normal Flow	1. Người thuê xác nhận trả xe. 2. Hệ thống cập nhật vị trí trả xe và tính tổng chi phí. 3. Thanh toán hoàn tất. 4. Hiện thị màn hình đánh giá.
Alt Flow	A1. Người thuê trả xe sớm hơn dự kiến → hệ thống điều chỉnh chi phí.
Exception Flow	E1. Không thể xác định vị trí xe → yêu cầu nhập vị trí thủ công. E2. Lỗi hệ thống thanh toán.
Extended Use Cases	Đánh giá sau thuê

UC-06 – Thanh toán

Trường	Mô tả
Actor	Người thuê xe
Description	Thực hiện thanh toán sau khi hoàn tất thuê xe hoặc đặt cọc trước khi thuê.
Trigger	Khi người dùng hoàn tất hành trình hoặc xác nhận đặt thuê.
Precondition	Người dùng đã chọn phương thức thanh toán (thẻ, ví điện tử, v.v.).
Postcondition	Giao dịch thanh toán thành công, lưu thông tin hóa đơn.
Normal Flow	1. Hệ thống tính toán chi phí thuê (thời gian × giá). 2. Hiện thị tổng tiền và yêu cầu xác nhận thanh toán. 3. Người dùng xác nhận. 4. Hệ thống kết nối cổng thanh toán (mock/payment gateway). 5. Trạng thái cập nhật thành "Paid".

Trường	Mô tả
Alt Flow	A1. Người thuê chọn thanh toán bằng điểm thưởng hoặc ví điện tử nội bộ. A2. Thanh toán tự động qua thẻ lưu sẵn.
Exception Flow	E1. Giao dịch thất bại → hoàn tác đặt xe. E2. Cổng thanh toán không phản hồi → thông báo "Thử lại sau".

UC-07 – Đánh giá sau thuê

Trường	Mô tả
Actor	Người thuê xe
Description	Người dùng để lại đánh giá và chấm sao cho xe/chủ xe.
Trigger	Sau khi trả xe hoặc từ trang "Lịch sử thuê".
Precondition	Đã hoàn tất thuê xe.
Postcondition	Đánh giá được lưu trữ và cập nhật điểm trung bình.
Normal Flow	1. Người dùng chọn số sao (1–5). 2. Nhập nội dung nhận xét. 3. Hệ thống lưu và cập nhật thông tin vào hồ sơ xe/chủ xe.
Alt Flow	A1. Người dùng chỉ đánh giá mà không viết bình luận.
Exception Flow	E1. Mạng lỗi, không gửi được đánh giá.
Included Use Cases	Rating, Review

UC-08 – Đăng ký xe cho thuê

Trường	Mô tả
Actor	Chủ xe (Owner)
Description	Cho phép chủ xe thêm xe điện của mình vào hệ thống để cho thuê.

Trường	Mô tả
Trigger	Chủ xe chọn "Đăng ký xe cho thuê" trong ứng dụng.
Precondition	Đã đăng nhập và xác thực danh tính.
Postcondition	Xe được thêm vào hệ thống với trạng thái "Chờ duyệt".
Normal Flow	1. Chủ xe nhập thông tin xe (biển số, model, dung lượng pin, giá, vị trí). 2. Hệ thống xác nhận dữ liệu hợp lệ. 3. Tạo bản ghi xe mới trong DB với trạng thái "pending". 4. Gửi thông báo cho admin để duyệt xe.
Alt Flow	A1. Chủ xe chọn lưu bản nháp để cập nhật sau. A2. Chủ xe tải lại ảnh hoặc cập nhật thông tin trước khi gửi.
Exception Flow	E1. Trùng biển số với xe khác → báo lỗi. E2. Ảnh không hợp lệ / sai định dạng. E3. Lỗi kết nối tới server.
Extended Use Cases	Duyệt xe (Admin duyệt để xe có thể hoạt động)

UC-09 – Quản lý xe của chủ

Trường	Mô tả
Actor	Chủ xe
Description	Quản lý danh sách xe đã đăng ký, cập nhật thông tin hoặc tình trạng sẵn sàng cho thuê.
Trigger	Chủ xe mở mục "Quản lý xe".
Precondition	Chủ xe đã có ít nhất 1 xe đăng ký.
Postcondition	Thông tin xe được cập nhật, đồng bộ lên hệ thống.

Trường	Mô tả
Normal Flow	- Hệ thống hiển thị danh sách xe của chủ. - Chủ xe chọn một xe → xem chi tiết. - Có thể cập nhật giá, trạng thái pin, vị trí, khả dụng (availability). - Hệ thống lưu và phản ánh thay đổi lên bản đồ.
Alt Flow	A1. Chủ xe tạm ẩn xe khỏi danh sách cho thuê. A2. Chủ xe cập nhật vị trí tự động qua GPS.
Exception Flow	E1. Xe đang cho thuê không thể bị xóa hoặc ẩn. E2. Cập nhật thất bại do mất kết nối.
Included Use Cases	Cập nhật vị trí, Tình trạng pin, Availability

UC-10 – Duyệt xe (Admin)

Trường	Mô tả
Actor	Quản trị viên
Description	Xem, kiểm duyệt và phê duyệt các xe được đăng ký bởi chủ xe.
Trigger	Admin đăng nhập vào bảng quản trị.
Precondition	Xe mới đăng ký có trạng thái "pending".
Postcondition	Xe được duyệt (status = "active") hoặc từ chối.
Normal Flow	- Admin mở danh sách xe chờ duyệt. - Xem thông tin, hình ảnh, giấy tờ. - Chọn "Phê duyệt" hoặc "Từ chối". - Hệ thống cập nhật trạng thái và thông báo cho chủ xe.
Alt Flow	A1. Admin yêu cầu bổ sung giấy tờ trước khi duyệt.
Exception Flow	E1. Dữ liệu xe không hợp lệ hoặc lỗi kết nối DB.

UC-11 – Xem báo cáo / thống kê

Trường	Mô tả
Actor	Quản trị viên
Description	Xem các thống kê về số lượng thuê, doanh thu, hoạt động người dùng.
Trigger	Admin chọn "Thống kê / Báo cáo".
Precondition	Có dữ liệu hoạt động trên hệ thống.
Postcondition	Báo cáo hiển thị trên dashboard.
Normal Flow	1. Admin chọn loại báo cáo (doanh thu, lượt thuê, hoạt động theo khu vực). 2. Hệ thống tổng hợp dữ liệu từ DB. 3. Hiển thị biểu đồ, bảng thống kê.
Alt Flow	A1. Admin chọn khoảng thời gian khác để xem. A2. Xuất báo cáo ra file PDF.
Exception Flow	E1. Dữ liệu chưa được cập nhật kịp thời. E2. Lỗi truy vấn SQL hoặc timeout.

UC-12 – Quản lý người dùng (Admin)

Trường	Mô tả
Actor	Quản trị viên
Description	Theo dõi, khóa, hoặc cấp quyền cho người dùng hệ thống.
Trigger	Admin truy cập mục "Quản lý người dùng".
Precondition	Admin đăng nhập thành công.
Postcondition	Người dùng bị cập nhật trạng thái hoặc quyền hạn.
Normal Flow	1. Hệ thống hiển thị danh sách người dùng. 2. Admin tìm kiếm theo tên/email. 3. Admin có thể khóa, mở khóa, hoặc thay đổi quyền.

Trường	Mô tả
Alt Flow	A1. Admin gửi thông báo cảnh cáo tới người dùng vi phạm. A2. Cấp quyền "Owner" cho người dùng mới.
Exception Flow	E1. Không có quyền đủ để sửa người dùng khác cấp cao hơn. E2. Lỗi truy cập cơ sở dữ liệu.

UC-13 – Đánh giá & Xếp hạng người dùng (Trust Score System)

Trường	Mô tả
Use Case Name	Đánh giá & Xếp hạng người dùng (Trust Score System)
Actor	Người thuê (Renter), Chủ xe (Owner), Hệ thống AI/Backend
Description	Sau khi hoàn tất giao dịch, cả hai bên (người thuê và chủ xe) có thể đánh giá lẫn nhau. Hệ thống tổng hợp các đánh giá để xây dựng chỉ số uy tín (Trust Score) của mỗi người dùng.
Trigger	Kết thúc giao dịch thuê xe (hoặc định kỳ hệ thống cập nhật lại Trust Score).
Precondition	Giao dịch đã hoàn tất (status = "completed").
Postcondition	Điểm Trust Score và đánh giá được cập nhật trong hồ sơ người dùng.
Normal Flow	- Sau khi hoàn tất chuyến thuê, hệ thống gửi yêu cầu đánh giá cho cả hai bên. - Người thuê chấm sao và để lại nhận xét về chủ xe (thái độ, tình trạng xe, đúng giờ). - Chủ xe chấm sao người thuê (cách sử dụng xe, đúng hẹn, hoàn trả đầy đủ). - Hệ thống lưu cả hai đánh giá, cập nhật Trust Score: Trust Score = average of (rating, dispute rate, activity, on-time return rate, verification level). - Điểm Trust Score hiển thị công khai trên hồ sơ người dùng.

Trường	Mô tả
Alt Flow	A1. Một bên bỏ qua đánh giá → hệ thống gửi nhắc sau 24h. A2. Người dùng báo cáo đánh giá sai lệch → admin can thiệp.
Exception Flow	E1. Mạng lỗi khi gửi đánh giá. E2. Hệ thống AI không tính được Trust Score do thiếu dữ liệu.
Included Use Cases	Rating, Review
Extended Use Cases	Phân tích uy tín bằng AI (tùy chọn mở rộng sau này)

Mô hình dữ liệu Trust Score (gợi ý)

Thuộc tính	Nguồn dữ liệu	Trọng số
Điểm trung bình đánh giá	Review của các counterparties	0.4
Tỷ lệ hoàn trả đúng giờ	Rental History	0.2
Tỷ lệ tranh chấp (dispute)	Admin report	-0.2
Xác thực tài khoản (KYC)	Verification API	0.1
Tần suất hoạt động / số giao dịch hoàn tất	Transaction logs	0.1

UC-15 – Xem lịch sử thuê / thu nhập

Trường	Mô tả
Actor	Người thuê / Chủ xe
Description	Xem danh sách các giao dịch thuê xe (đã, đang, hoặc sắp tới). Chủ xe có thể xem thu nhập.
Trigger	Người dùng chọn "Lịch sử thuê" hoặc "Doanh thu".
Precondition	Đã đăng nhập.

Trường	Mô tả
Postcondition	Hiển thị danh sách lịch sử có thể lọc theo thời gian.
Normal Flow	- Gửi yêu cầu tới server <code>/rentals?userId=</code>. - Hệ thống trả về danh sách và tổng thu nhập (đối với chủ xe). - Người dùng có thể xem chi tiết từng giao dịch.
Alt Flow	A1. Lọc theo ngày, khu vực, loại xe. A2. Xuất dữ liệu thành file CSV/PDF.
Exception Flow	E1. Không có dữ liệu phù hợp → hiển thị thông báo. E2. Lỗi DB hoặc timeout.
Extended Use Cases	Xem thu nhập (dành riêng cho chủ xe)

UC-14 – Gợi ý thời điểm thuê (AI Module)

Trường	Mô tả
Actor	Người thuê xe, Hệ thống AI
Description	Hệ thống gợi ý thời gian thuê phù hợp dựa trên dữ liệu lịch sử và hành vi người dùng.
Trigger	Người dùng chọn "Gợi ý thời điểm thuê".
Precondition	Hệ thống có dữ liệu lịch sử giao dịch.
Postcondition	Gợi ý hiển thị trên giao diện.
Normal Flow	- Người dùng yêu cầu gợi ý. - Backend gửi dữ liệu lịch sử cho mô hình ML. - ML xử lý và trả về khung giờ gợi ý cùng độ tin cậy (confidence). - Hệ thống hiển thị cho người dùng.
Alt Flow	A1. Nếu chưa có dữ liệu lịch sử → hiển thị khuyến nghị mặc định (rule-based).

Trường	Mô tả
Exception Flow	E1. AI service tạm ngưng → hiển thị lỗi "Dịch vụ dự báo không khả dụng".

UC-16 – Báo cáo sự cố (Incident Report)

Trường	Mô tả
Actor	Người thuê (Renter), Chủ xe (Owner), Quản trị viên (Admin)
Mục tiêu	Cho phép người dùng báo cáo sự cố trong quá trình thuê xe hoặc phát hiện vấn đề với phương tiện.
Trigger	Người dùng chọn "Báo cáo sự cố" trong giao diện "Theo dõi chuyến đi" hoặc sau khi hoàn tất thuê.
Precondition	Có chuyến thuê đang hoạt động hoặc vừa hoàn tất.
Postcondition	Báo cáo được lưu vào hệ thống và chờ Admin xử lý.
Normal Flow	1. Người thuê mở mục "Báo cáo sự cố". 2. Chọn loại sự cố (xe không hoạt động, tai nạn, hư pin, mất xe, v.v.). 3. Nhập mô tả chi tiết, kèm ảnh hoặc video. 4. Hệ thống gửi dữ liệu đến server. 5. Admin nhận thông báo và phản hồi qua trung tâm quản trị để xử lý.
Alternative Flow	A1. Chủ xe báo cáo sự cố do người thuê gây ra (ví dụ: trả xe không đúng vị trí, hư hỏng xe). A2. Người thuê gửi thêm bằng chứng bổ sung sau khi tạo báo cáo.
Exception Flow	E1. Không tải được ảnh hoặc lỗi mạng → lưu tạm cục bộ, gửi lại khi có kết nối. E2. Admin chưa phản hồi trong thời hạn → hệ thống gửi nhắc tự động.
Included Use Cases	Thông báo (Notification)

Trường	Mô tả
Extended Use Cases	Xem lịch sử thuê (UC-15)

UC-17 – Notification System

Trường	Mô tả
Actor	Người thuê (Renter), Chủ xe (Owner), Quản trị viên (Admin), Hệ thống Notification Service
Mục tiêu	Gửi và hiển thị thông báo (real-time hoặc offline) khi có sự kiện liên quan trong hệ thống.
Trigger	Một sự kiện xảy ra trong hệ thống (ví dụ: thuê xe, trả xe, phê duyệt xe, đánh giá, báo cáo sự cố).
Precondition	Người dùng có tài khoản hợp lệ và đăng nhập vào hệ thống.
Postcondition	Thông báo được gửi đến đúng đối tượng và lưu lại trong lịch sử thông báo.

Trường	Mô tả
Normal Flow	- Một hành động hoặc sự kiện diễn ra, ví dụ: - Người thuê gửi yêu cầu thuê xe (UC-04). - Admin duyệt xe (UC-10). - Người thuê đánh giá chủ xe (UC-07). - Có báo cáo sự cố (UC-16). - Backend tạo event. - Notification Service nhận event từ hệ thống chính qua message queue (hoặc event bus). - Tùy theo loại sự kiện, hệ thống xác định danh sách người nhận: - Renter: thông báo xác nhận thuê / phản hồi từ Owner. - Owner: thông báo khi có người muốn thuê / khi xe được trả. - Admin: thông báo có sự cố hoặc cần duyệt xe. - Gửi thông báo: - Real-time (push): thông qua Firebase Cloud Messaging hoặc WebSocket. - Offline (inbox): lưu trong bảng <code>notifications</code> để xem lại sau. - Người dùng nhận thông báo trên ứng dụng (popup, icon badge, hoặc email).
Alternative Flow	- Người dùng tắt thông báo đẩy → Hệ thống chỉ lưu thông báo trong hộp thư (inbox), không push. - Người dùng đã đọc thông báo → Trạng thái thông báo được cập nhật thành <code>read = true</code>. - Người dùng xóa thông báo → Thông báo được ẩn khỏi giao diện nhưng vẫn lưu lịch sử trong DB.
Exception Flow	- Firebase / WebSocket mất kết nối → Thông báo sẽ được gửi lại khi người dùng online hoặc gửi email dự phòng. - Event payload không hợp lệ → Notification Service ghi log lỗi và bỏ qua sự kiện. - Người nhận không tồn tại hoặc đã bị khóa → Bỏ qua hoặc gửi thông báo tới Admin để kiểm tra.

5.2 Mô hình hoá quy trình nghiệp vụ

5.2.1 Quy trình thuê xe (UC-02, UC-03, UC-04)

(Hình ??)

Mục tiêu:

Cho phép người dùng tìm kiếm, xem thông tin và đặt thuê xe máy điện trong khu vực mong muốn thông qua ứng dụng.

Actor chính: Người thuê (Renter)

Các bước chính:

1. Người thuê đăng nhập và chọn chức năng “Tìm xe gần tôi” hoặc nhập vị trí thủ công.
2. Hệ thống xác định vị trí, truy vấn và hiển thị danh sách xe khả dụng.
3. Người thuê chọn xe để xem thông tin chi tiết (giá, pin, vị trí, chủ xe, đánh giá).
4. Người thuê chọn “Đặt thuê”, nhập thời gian và xác nhận.
5. Hệ thống gợi ý thời điểm thuê tối ưu (AI), kiểm tra tình trạng xe, tạo đơn thuê và gửi thông báo đến chủ xe.
6. Người thuê nhận xác nhận thành công, hoàn tất quá trình thuê nếu chủ xe xác nhận cho thuê.

Luồng thay thế:

Người dùng có thể nhập vị trí thủ công khi không bật GPS. Nếu không có xe khả dụng, hệ thống gợi ý mở rộng phạm vi hoặc thời gian khác.

5.2.2 Quy trình trả xe (UC-05, UC-06, UC-07)

(Hình ??)

Mục tiêu:

Cho phép người thuê kết thúc hành trình, trả xe và thực hiện thanh toán, đồng thời gửi đánh giá cho chủ xe hoặc phương tiện sau khi hoàn tất chuyến đi.

Actor chính: Người thuê (Renter)

Các bước chính:

1. Người thuê chọn chức năng “Trả xe” trong ứng dụng.
2. Hệ thống xác nhận vị trí hiện tại của xe hoặc yêu cầu nhập thủ công nếu không định vị được.
3. Hệ thống tính toán chi phí thuê thực tế dựa trên thời gian và quãng đường.
4. Người thuê xác nhận thanh toán.
5. Sau khi thanh toán thành công, hệ thống cập nhật trạng thái xe thành “Available”.
6. Ứng dụng hiển thị màn hình đánh giá xe và chủ xe.
7. Người thuê gửi đánh giá (số sao, nội dung) để hoàn tất quy trình.

Luồng thay thế:

- Nếu người thuê trả xe sớm hơn thời gian dự kiến, hệ thống tự động điều chỉnh chi phí.
- Nếu GPS không xác định được vị trí, hệ thống yêu cầu người dùng nhập vị trí trả xe thủ công.

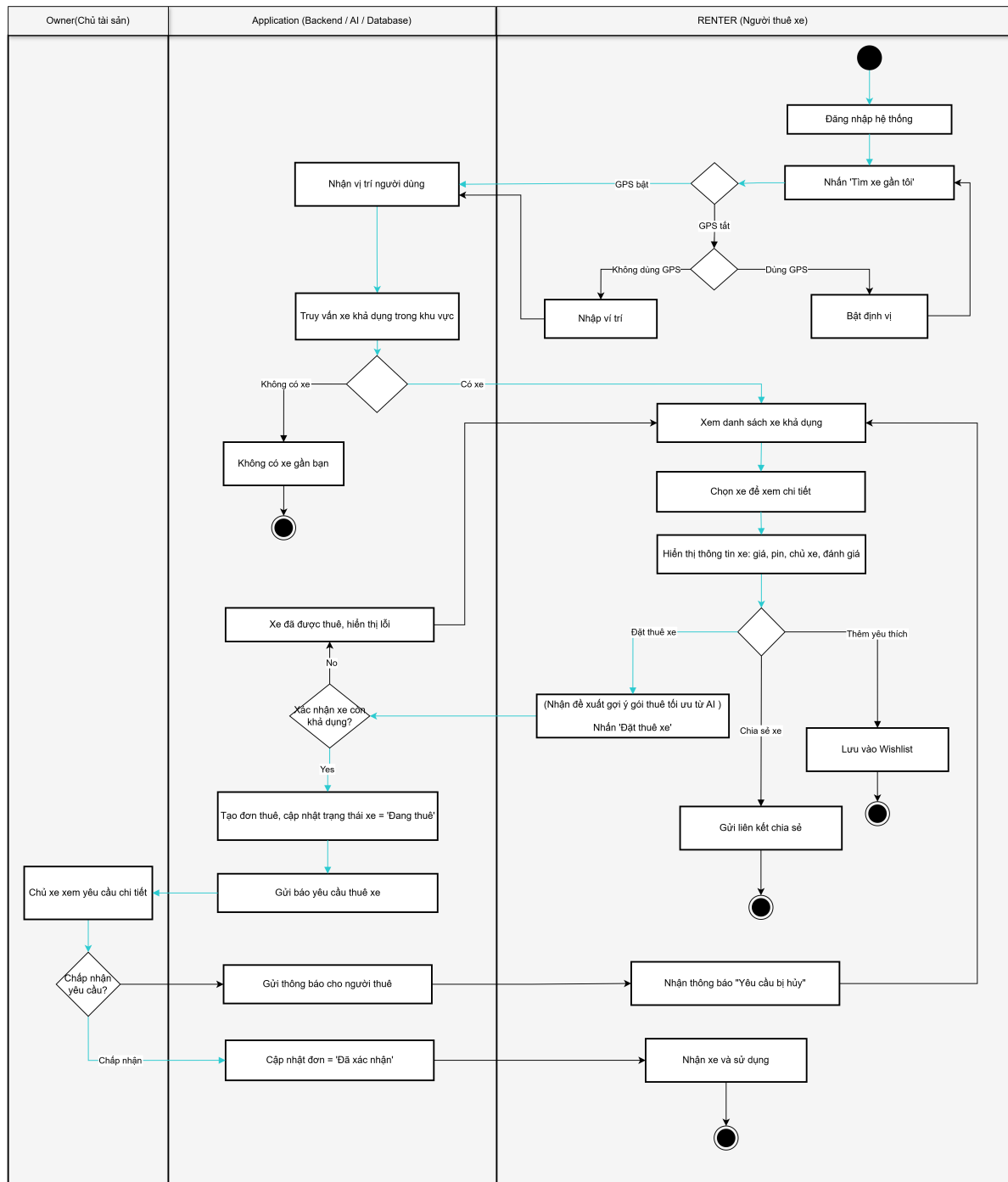


Figure 5: *Renting Activity Diagram*

Mục tiêu:

Cho phép chủ xe thêm phương tiện của mình vào hệ thống để chia sẻ hoặc cho thuê, đảm bảo thông tin được xác thực và duyệt bởi quản trị viên trước khi hoạt động.

Actor chính: Chủ xe (Owner)

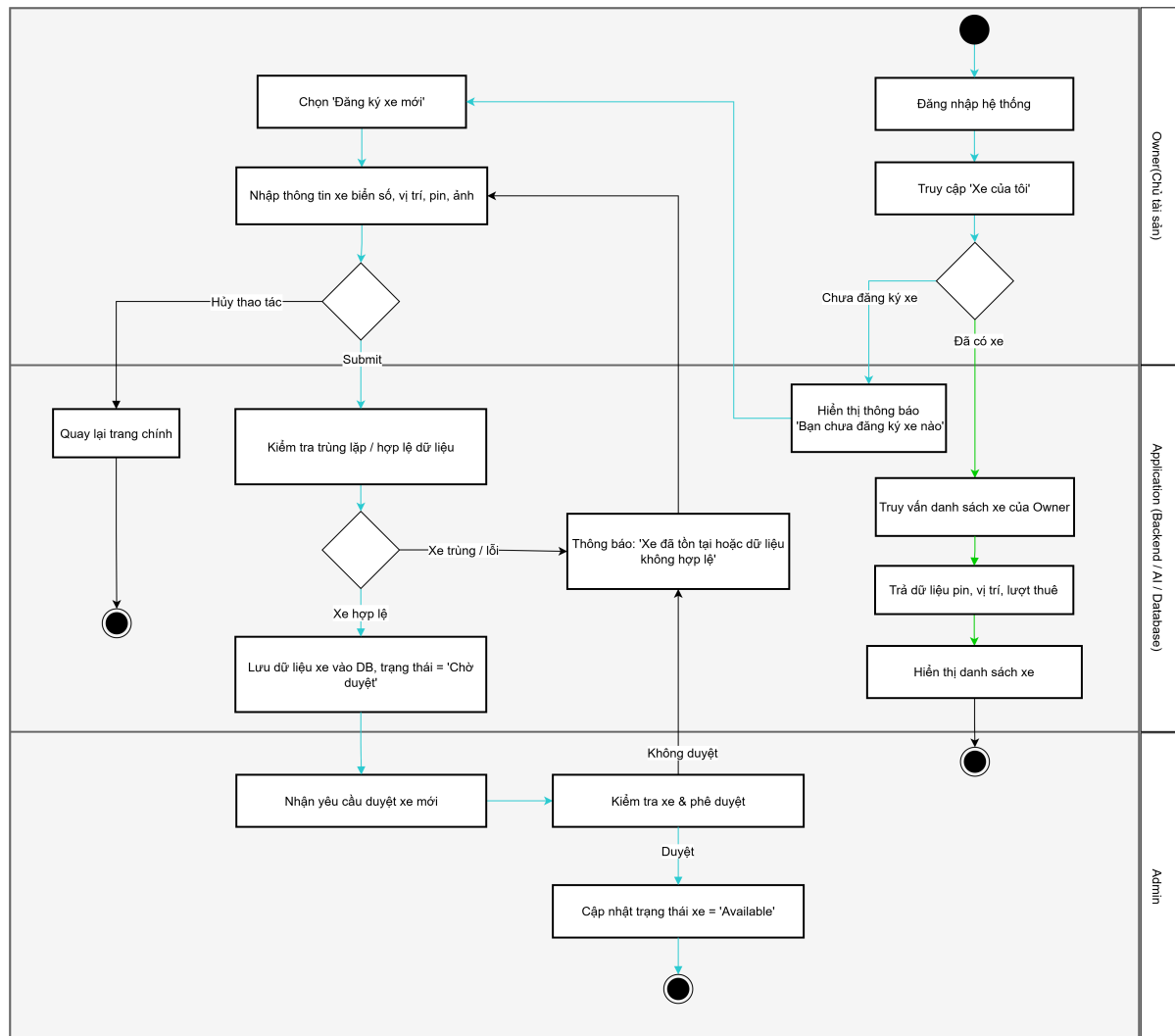


Figure 7: Register Vehicle Activity Diagram

Các bước chính:

- Chủ xe đăng nhập và chọn chức năng “Đăng ký xe cho thuê”.
- Hệ thống hiển thị biểu mẫu để nhập thông tin xe: biển số, model, dung lượng pin, giá thuê, vị trí và hình ảnh.
- Chủ xe nhập thông tin và gửi yêu cầu đăng ký.
- Hệ thống kiểm tra dữ liệu hợp lệ (trùng biển số, định dạng ảnh, thông tin bắt buộc).
- Nếu hợp lệ, hệ thống tạo bản ghi xe mới trong cơ sở dữ liệu với trạng thái “Chờ duyệt (Pending)”.

6. Gửi thông báo đến quản trị viên để xem xét và phê duyệt.
7. Sau khi được duyệt, xe chuyển sang trạng thái “Available” và sẵn sàng cho thuê.

Luồng thay thế:

- Chủ xe có thể lưu bản nháp nếu chưa hoàn tất thông tin.
- Nếu phát hiện lỗi dữ liệu (trùng biển số, ảnh sai định dạng), hệ thống hiển thị thông báo để người dùng chỉnh sửa.

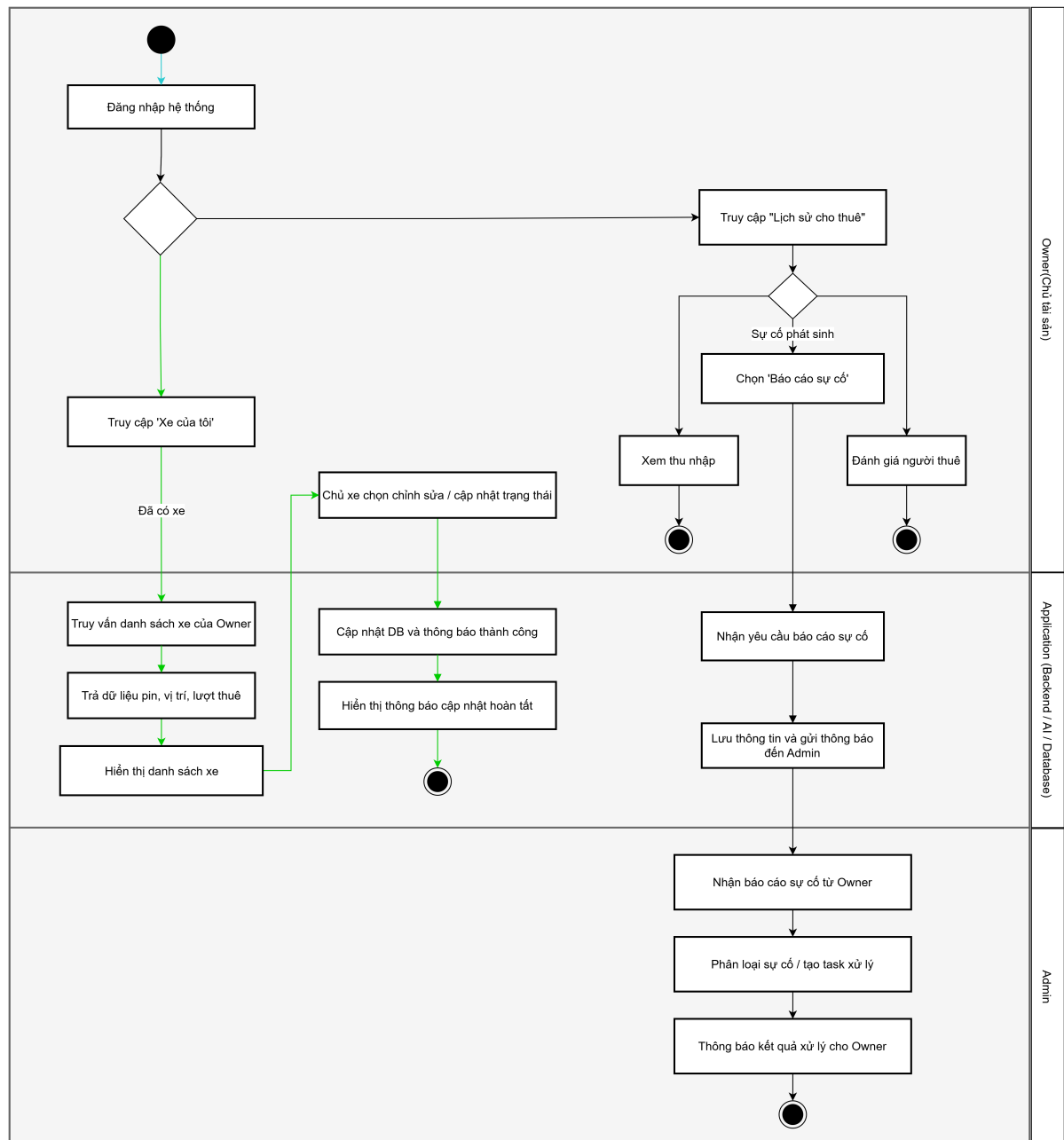


Figure 8: Manage Vehicle Activity Diagram

5.2.4 Quy trình quản lý xe của chủ (UC-09)

(Hình ??)

Mục tiêu:

Cho phép chủ xe theo dõi, cập nhật và quản lý các xe đã đăng ký cho thuê, đảm bảo thông tin xe luôn chính xác và đồng bộ với hệ thống.

Actor chính: Chủ xe (Owner)

Các bước chính:

1. Chủ xe đăng nhập và chọn mục “Quản lý xe”.
2. Hệ thống hiển thị danh sách tất cả xe mà chủ xe đã đăng ký.
3. Chủ xe chọn một xe để xem thông tin chi tiết.
4. Thực hiện các thao tác: cập nhật giá, trạng thái pin, vị trí, hoặc tạm ẩn xe khỏi danh sách cho thuê.
5. Hệ thống xác nhận và lưu thay đổi vào cơ sở dữ liệu.
6. Các thay đổi được đồng bộ và phản ánh lên bản đồ xe cho thuê.

Luồng thay thế:

- Chủ xe có thể kích hoạt cập nhật vị trí tự động qua GPS.
- Xe đang cho thuê không thể bị ẩn hoặc xóa khỏi hệ thống cho đến khi kết thúc hành trình.

5.2.5 Quy trình quản trị hệ thống (UC-10, UC-11, UC-12)

(Hình ??)

Mục tiêu:

Đảm bảo nền tảng hoạt động ổn định, an toàn và minh bạch thông qua việc kiểm duyệt xe, quản lý người dùng, và theo dõi hiệu quả hoạt động chung của hệ thống.

Actor chính: Quản trị viên (Admin)

Các bước chính:

1. Admin đăng nhập vào hệ thống quản trị.
2. Hệ thống hiển thị bảng điều khiển (Dashboard) với các module: *Duyệt xe*, *Quản lý người dùng*, *Báo cáo - Thống kê*.
3. Admin chọn module cần thao tác:
 - 3.1. **Duyệt xe:** Xem danh sách xe chờ duyệt, kiểm tra thông tin, hình ảnh, giấy tờ → phê duyệt hoặc từ chối → hệ thống cập nhật trạng thái và gửi thông báo cho chủ xe.
 - 3.2. **Quản lý người dùng:** Xem danh sách người dùng, khóa/mở khóa tài khoản, thay đổi quyền truy cập (User, Owner, Admin), gửi cảnh cáo cho người vi phạm.
 - 3.3. **Báo cáo - Thống kê:** Chọn loại báo cáo (doanh thu, lượt thuê, hoạt động theo khu vực),

hệ thống tổng hợp dữ liệu, hiển thị biểu đồ và cho phép xuất file (PDF, Excel).

4. Sau khi hoàn tất thao tác, hệ thống lưu lại log quản trị (audit trail) để theo dõi thay đổi.

5. Quản trị viên có thể đăng xuất hoặc chuyển sang module khác mà không ảnh hưởng dữ liệu.

Luồng thay thế:

- Nếu dữ liệu bị lỗi hoặc chậm cập nhật, hệ thống hiển thị thông báo “Dữ liệu đang đồng bộ”.
- Khi không đủ quyền thao tác, hiển thị cảnh báo “Không đủ quyền truy cập”.
- Admin có thể lọc dữ liệu hoặc thay đổi thời gian xem thống kê linh hoạt.

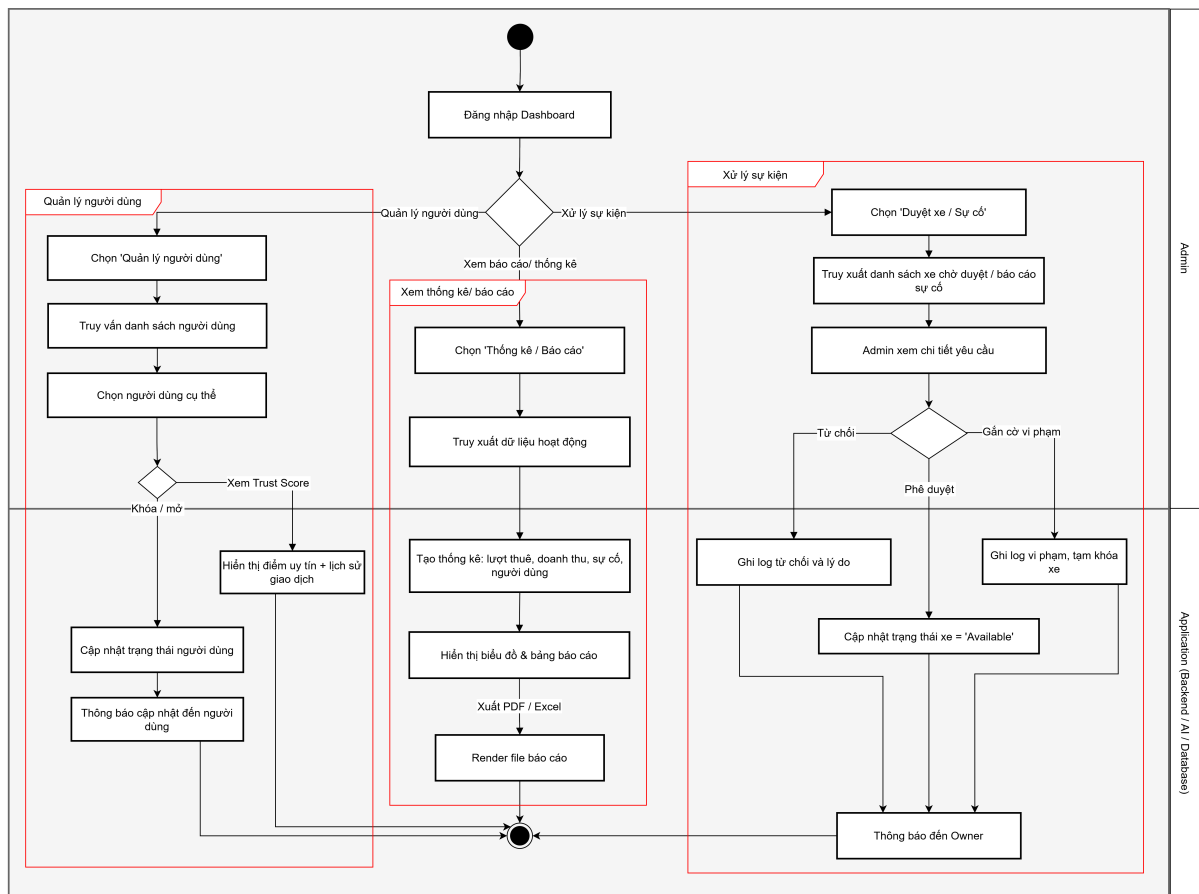


Figure 9: Administrative Action Activity Diagram

5.3 Mô hình hóa hành vi hệ thống

Renter Flow

5.3.1 Tìm và Đặt thuê xe (UC-02, UC-03, UC-04) (Hình ??)

Mô tả tổng quát

Người thuê tìm kiếm xe điện gần vị trí hiện tại, xem chi tiết và tiến hành đặt thuê.

Thành phần và vai trò

1. **Renter:** Khởi tạo yêu cầu tìm xe, xem chi tiết, đặt thuê
2. **MobileApp:** Giao diện và trung gian gửi yêu cầu
3. **RentalController:** Xử lý logic chính, điều phối các dịch vụ
4. **VehicleService:** Cung cấp dữ liệu xe khả dụng
5. **NotificationService:** Gửi thông báo xác nhận cho chủ xe
6. **Database:** Lưu trữ thông tin xe và đơn thuê

Hành vi hệ thống

1. Người thuê chọn "Tìm xe gần tôi", app gửi yêu cầu vị trí GPS.
2. RentalController gọi VehicleService truy vấn danh sách xe khả dụng trong bán kính.
3. Kết quả hiển thị trên bản đồ, người thuê chọn 1 xe để xem chi tiết.
4. Khi nhấn "Đặt thuê", hệ thống kiểm tra tình trạng xe, ghi nhận đơn trong Database, gửi notification đến chủ xe.
5. Nếu xe bận hoặc mạng lỗi → hiển thị thông báo lỗi.
6. Người thuê có thể chỉnh sửa thời gian thuê trước khi chủ xe xác nhận.

5.3.2 Theo dõi chuyển đi

Mô tả tổng quát

Hệ thống giám sát hành trình, pin và chi phí theo thời gian thực trong khi xe đang được thuê.

Thành phần và vai trò

1. **Renter:** Theo dõi hành trình
2. **MobileApp:** Hiển thị bản đồ, đồng bộ dữ liệu
3. **RentalController:** Xử lý truy vấn trạng thái
4. **VehicleService:** Cập nhật thông tin xe
5. **Database:** Lưu dữ liệu vị trí, pin, chi phí

Hành vi hệ thống

1. Sau khi chủ xe chấp nhận, booking chuyển sang active.
2. App gửi yêu cầu cập nhật trạng thái định kỳ (polling mỗi 10s).

3. Hệ thống lấy dữ liệu vị trí, pin, thời gian, chi phí từ DB.
4. App hiển thị hành trình theo thời gian thực.
5. Khi pin $< 20\%$ → gửi cảnh báo cho người dùng.
6. Nếu lỗi GPS → cho phép nhập vị trí thủ công.

5.3.3 Kết thúc chuyến đi & Đánh giá (UC-05, UC-06, UC-07) (Hình ??)

Mô tả tổng quát

Người thuê kết thúc hành trình, hệ thống thực hiện thanh toán tự động và hiển thị giao diện đánh giá chủ xe.

Thành phần và vai trò

1. **Renter:** Kết thúc chuyến đi và thực hiện đánh giá
2. **MobileApp:** Giao diện thanh toán và đánh giá
3. **RentalController:** Quản lý kết thúc thuê và xử lý logic
4. **VehicleService:** Cập nhật trạng thái xe về khả dụng
5. **PaymentService:** Tính toán và xử lý thanh toán
6. **NotificationService:** Gửi thông báo hoàn tất và cập nhật TrustScore
7. **Database:** Lưu thanh toán và đánh giá

Hành vi hệ thống

1. Renter nhấn "Kết thúc chuyến đi".
2. RentalController cập nhật trạng thái xe về available qua VehicleService.
3. PaymentService tính chi phí dựa trên thời gian và mức giá, thực hiện thanh toán.
4. Nếu thanh toán thành công → lưu kết quả và gửi thông báo hoàn tất đến Renter & Owner.
5. App hiển thị giao diện đánh giá sau thuê.
6. Người thuê có thể:
 - (a) Chấm sao và viết bình luận → lưu vào DB, cập nhật TrustScore.
 - (b) Chỉ chấm sao → lưu tối giản.
 - (c) Bỏ qua → đánh dấu review pending.
7. Nếu mất mạng → dữ liệu được lưu tạm và gửi lại khi có kết nối.

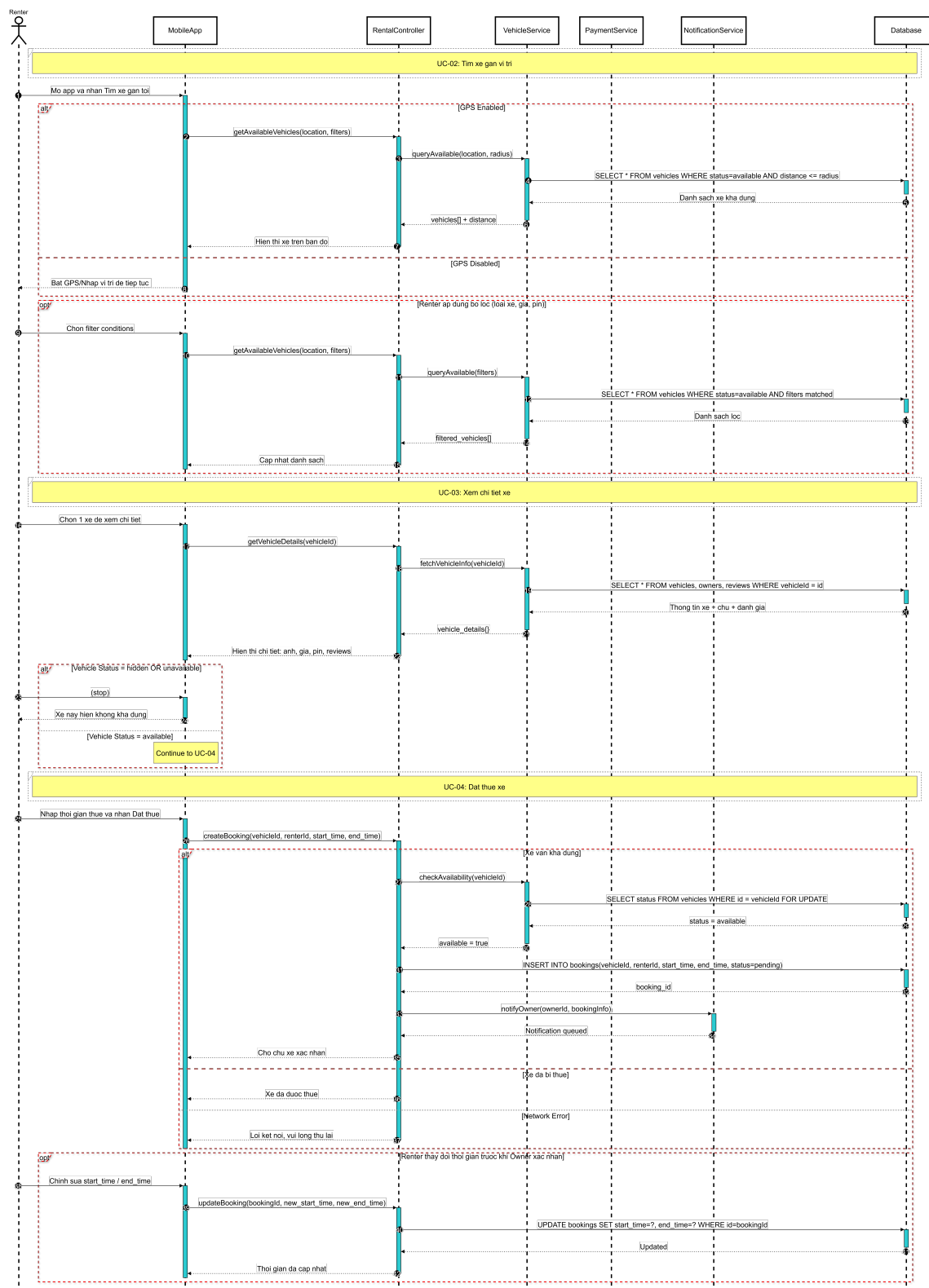


Figure 10: Sequence Diagram - Tìm và Đặt thuê xe

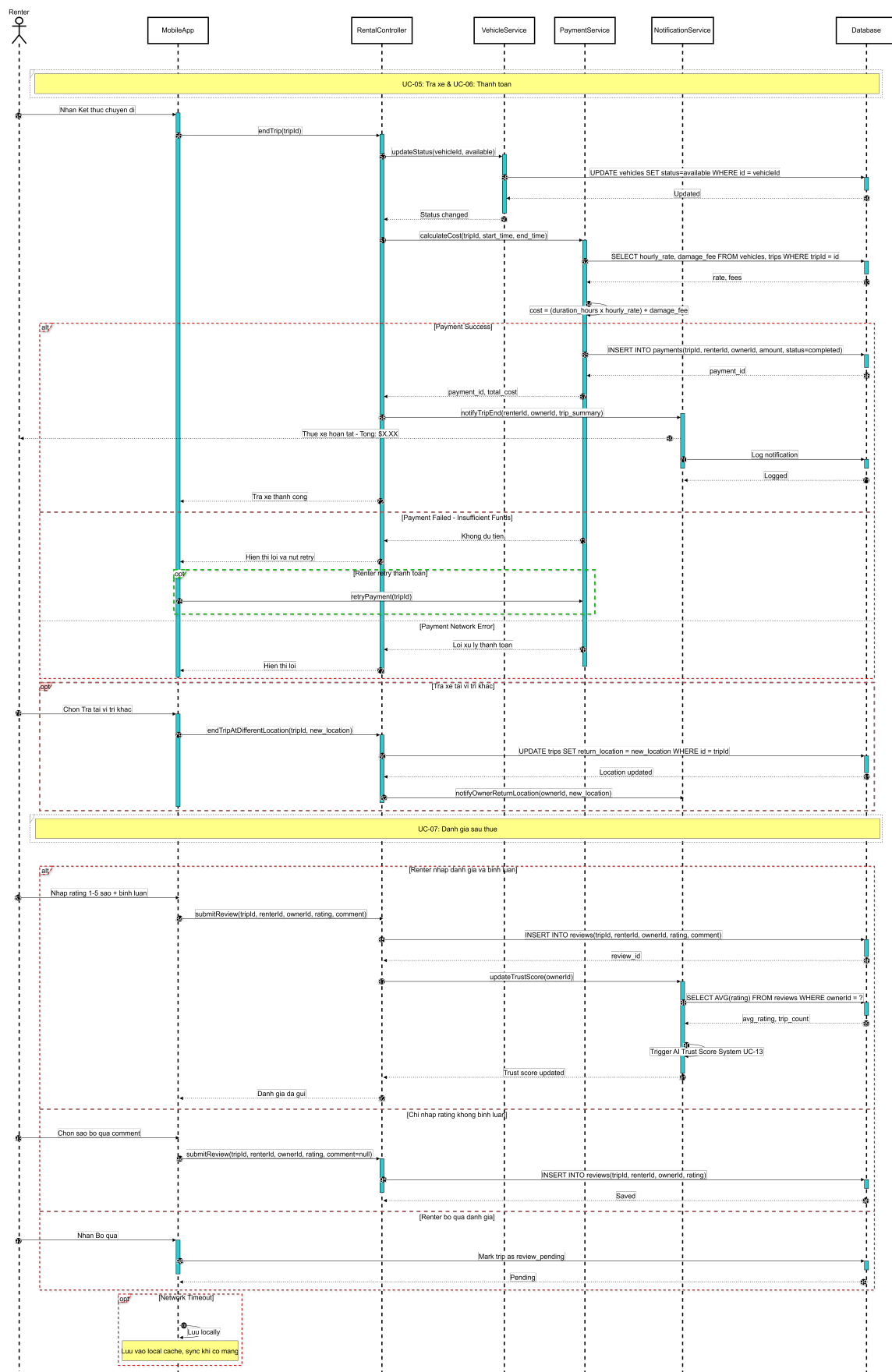


Figure 11: Sequence Diagram - Kết thúc chuyến đi & Đánh giá

Owner Flow

5.3.4 Đăng ký xe cho thuê (UC-08) (Hình ??)

Mô tả tổng quát

Chức năng cho phép chủ xe đăng ký phương tiện mới để đưa lên hệ thống cho thuê. Quá trình này yêu cầu xác nhận từ Admin trước khi xe được kích hoạt.

Thành phần và vai trò

1. **Owner:** Nhập thông tin và gửi yêu cầu đăng ký xe
2. **OwnerApp:** Giao diện nhập và gửi dữ liệu xe
3. **VehicleController:** Xử lý yêu cầu đăng ký, giao tiếp với DB và hệ thống duyệt
4. **Database:** Lưu thông tin xe với trạng thái "pending"
5. **AdminSystem:** Duyệt hoặc từ chối xe đăng ký
6. **NotificationService:** Gửi thông báo về kết quả duyệt

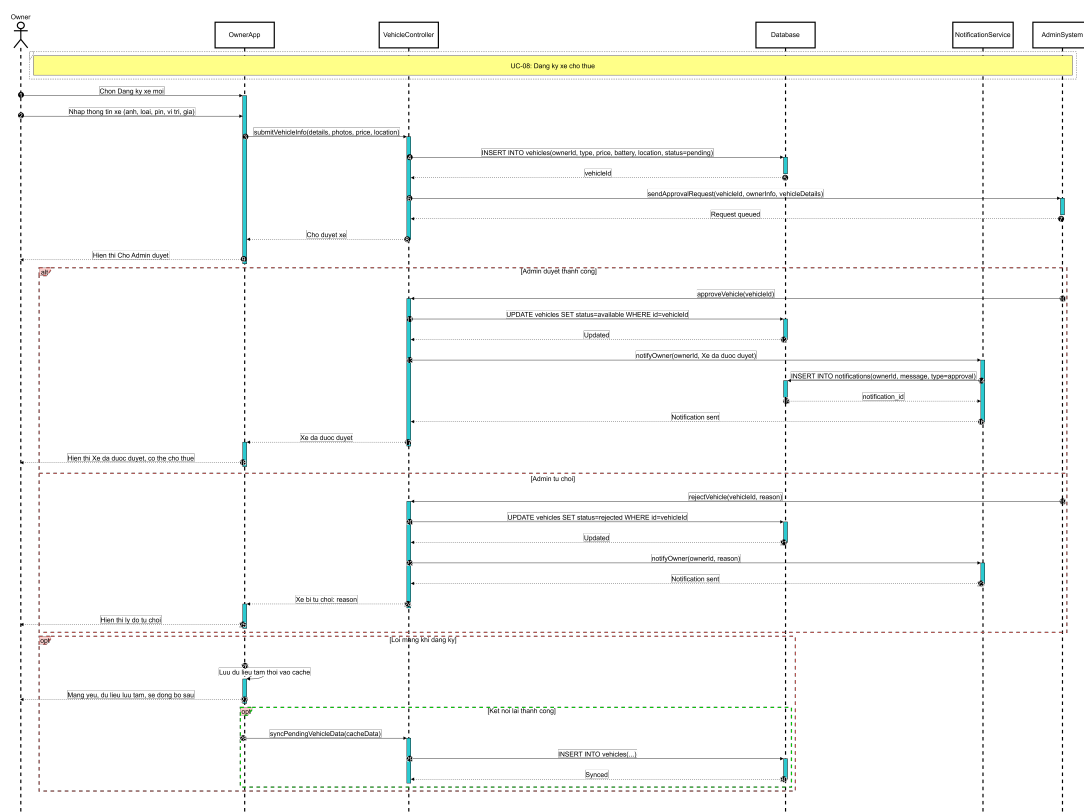


Figure 12: Sequence Diagram - Đăng ký xe cho thuê

Hành vi hệ thống

1. Owner nhập thông tin xe (ảnh, vị trí, giá, loại, pin).
2. OwnerApp gửi yêu cầu đến VehicleController → ghi vào Database với trạng thái pending.
3. VehicleController gửi yêu cầu duyệt đến AdminSystem.

4. Khi Admin phê duyệt: hệ thống cập nhật trạng thái xe thành available và NotificationService gửi thông báo "Xe đã được duyệt".
5. Nếu bị từ chối: hiển thị lý do cho Owner.
6. Nếu lỗi mạng khi gửi, dữ liệu được lưu tạm và đồng bộ khi có kết nối lại.

5.3.5 Quản lý xe (UC-09) (Hình ??)

Mô tả tổng quát

Cho phép chủ xe xem, cập nhật và theo dõi tình trạng xe trong danh mục của mình.

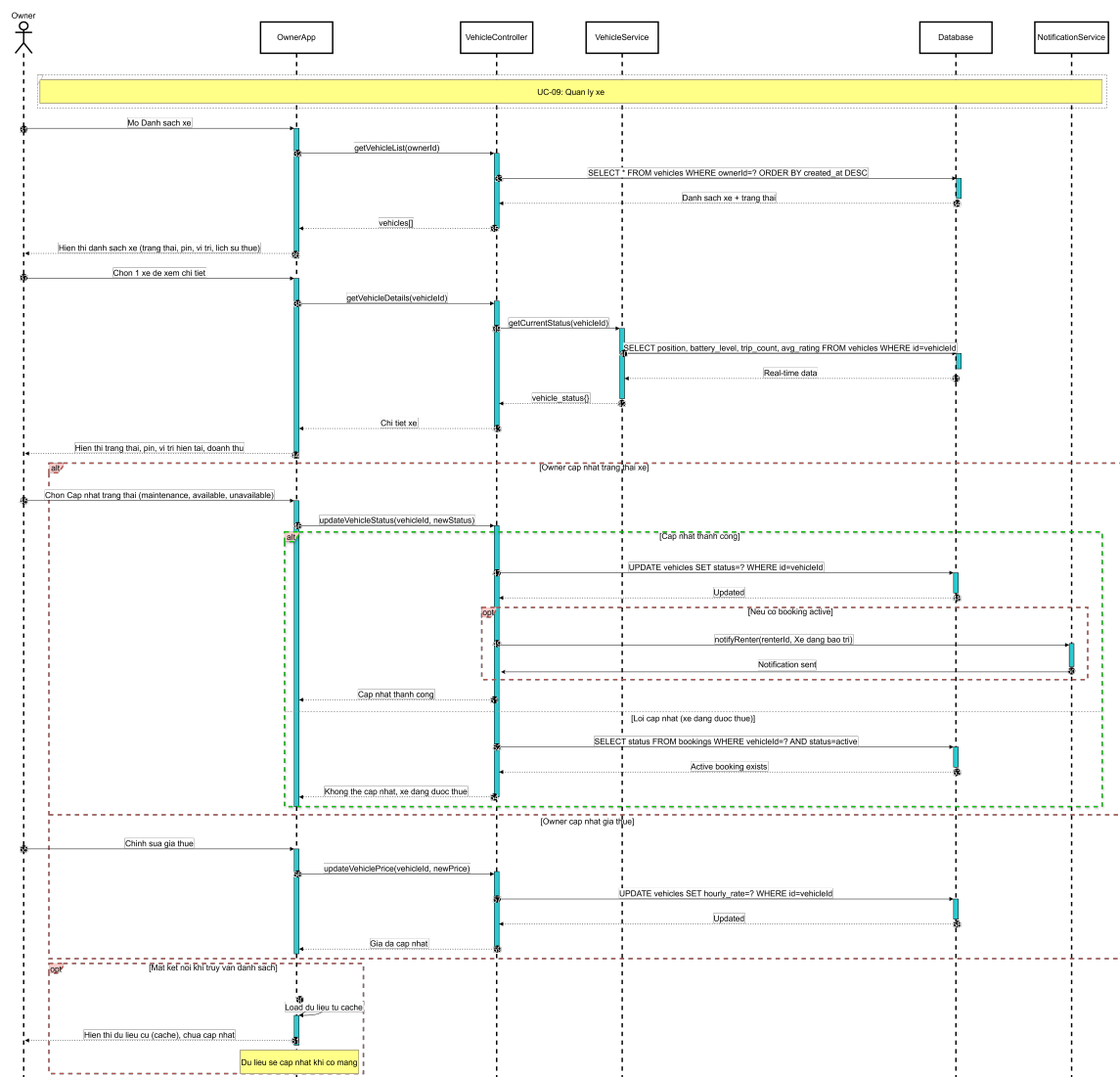


Figure 13: Sequence Diagram - Quản lý xe

Thành phần và vai trò

1. **Owner:** Xem danh sách xe, chỉnh sửa thông tin
2. **OwnerApp:** Hiển thị danh sách và giao diện cập nhật
3. **VehicleController:** Điều phối truy vấn và cập nhật dữ liệu

4. **VehicleService:** Cung cấp trạng thái xe (pin, vị trí, đánh giá)
5. **Database:** Lưu và cập nhật thông tin xe
6. **NotificationService:** Thông báo cho Renter nếu có thay đổi ảnh hưởng

Hành vi hệ thống

1. OwnerApp gửi yêu cầu xem danh sách xe → VehicleController truy vấn DB.
2. Owner có thể chọn xem chi tiết 1 xe → lấy dữ liệu từ VehicleService (pin, vị trí, điểm đánh giá).
3. Khi Owner cập nhật trạng thái hoặc giá:
 - Nếu hợp lệ → DB cập nhật thành công và hiển thị thông báo.
 - Nếu xe đang được thuê → từ chối thay đổi, hiển thị cảnh báo.
4. Nếu có booking đang hoạt động, NotificationService gửi thông báo đến Renter.
5. Khi mất kết nối, app hiển thị dữ liệu cache cũ và đồng bộ khi có mạng.

5.3.6 Báo cáo sự cố (UC-16) (Hình ??)

Mô tả tổng quát

Hỗ trợ chủ xe báo cáo vấn đề liên quan đến xe, hành vi thuê, hoặc sự cố kỹ thuật.

Thành phần và vai trò

1. **Owner:** Tạo báo cáo sự cố
2. **OwnerApp:** Gửi yêu cầu báo cáo
3. **VehicleController:** Ghi nhận và gửi báo cáo đến Admin
4. **Database:** Lưu trạng thái và log báo cáo
5. **AdminSystem:** Tiếp nhận và xác nhận xử lý
6. **NotificationService:** Gửi phản hồi lại cho chủ xe

Hành vi hệ thống

1. Owner nhập mô tả sự cố và gửi đến VehicleController.
2. Hệ thống lưu báo cáo (pending) và gửi lên AdminSystem để xử lý.
3. Khi Admin phản hồi → cập nhật trạng thái acknowledged trong DB và thông báo cho Owner.
4. Nếu timeout hoặc không phản hồi → hiển thị lỗi, cho phép gửi lại.

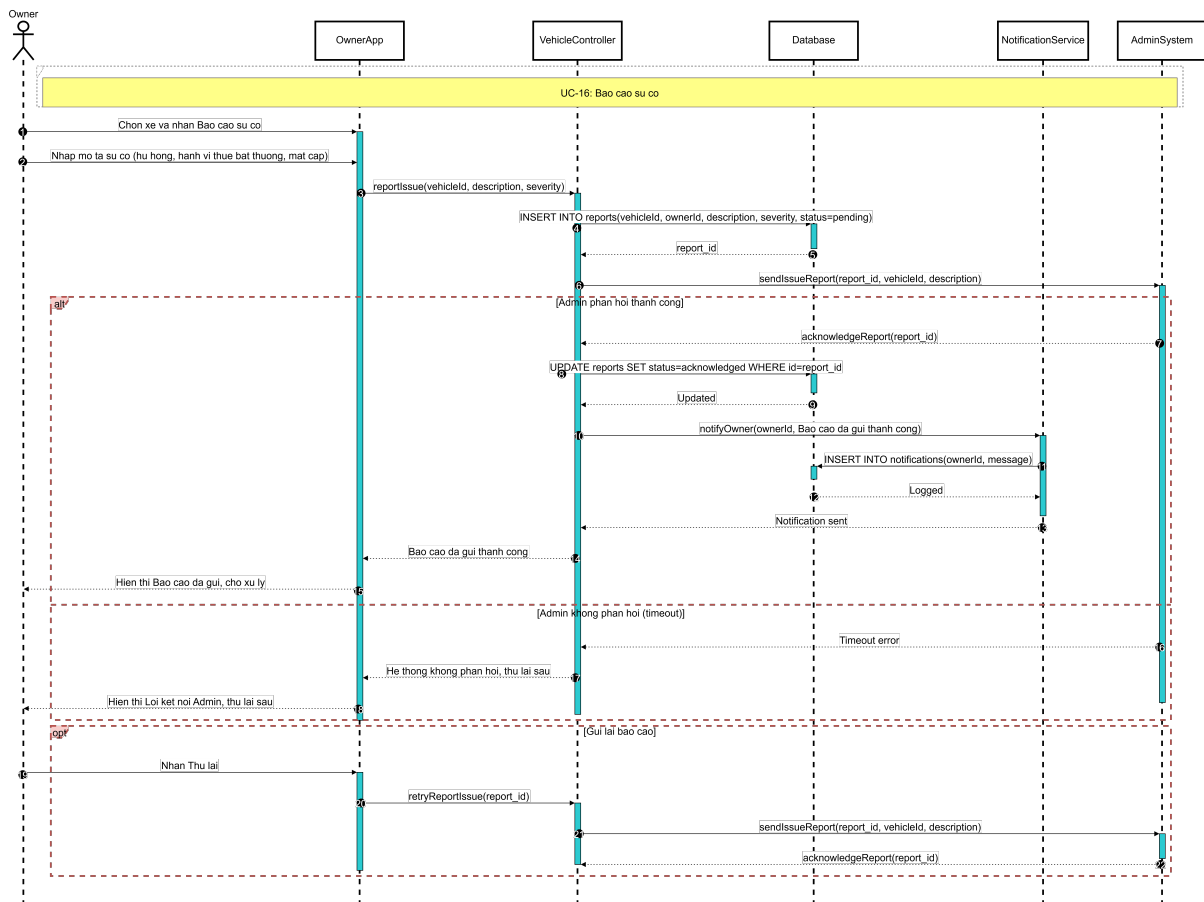


Figure 14: Sequence Diagram - Báo cáo sự cố

5.3.7 Thông báo hệ thống (UC-17) (Hình ??)

Mô tả tổng quát

Các thông báo push được gửi tự động từ NotificationService đến OwnerApp khi có sự kiện quan trọng.

Thành phần và vai trò

1. **NotificationService:** Phát sinh và gửi thông báo
2. **OwnerApp:** Hiển thị thông báo, xử lý phản hồi
3. **VehicleController:** Cập nhật trạng thái khi Owner phản hồi
4. **AdminSystem:** Xử lý vi phạm hoặc phản hồi Owner
5. **Database:** Lưu log và hàng đợi thông báo

Hành vi hệ thống

1. Khi có sự kiện như xe được duyệt, yêu cầu thuê mới, hoặc báo cáo được xử lý, NotificationService gửi push đến OwnerApp.
2. OwnerApp hiển thị nội dung thông báo và cho phép phản hồi.
3. Khi Owner phản hồi (ví dụ xác nhận booking), VehicleController cập nhật DB và gửi thông

báo ngược lại cho Renter.

4. Nếu lỗi kết nối → NotificationService lưu thông báo vào queue để gửi lại sau.

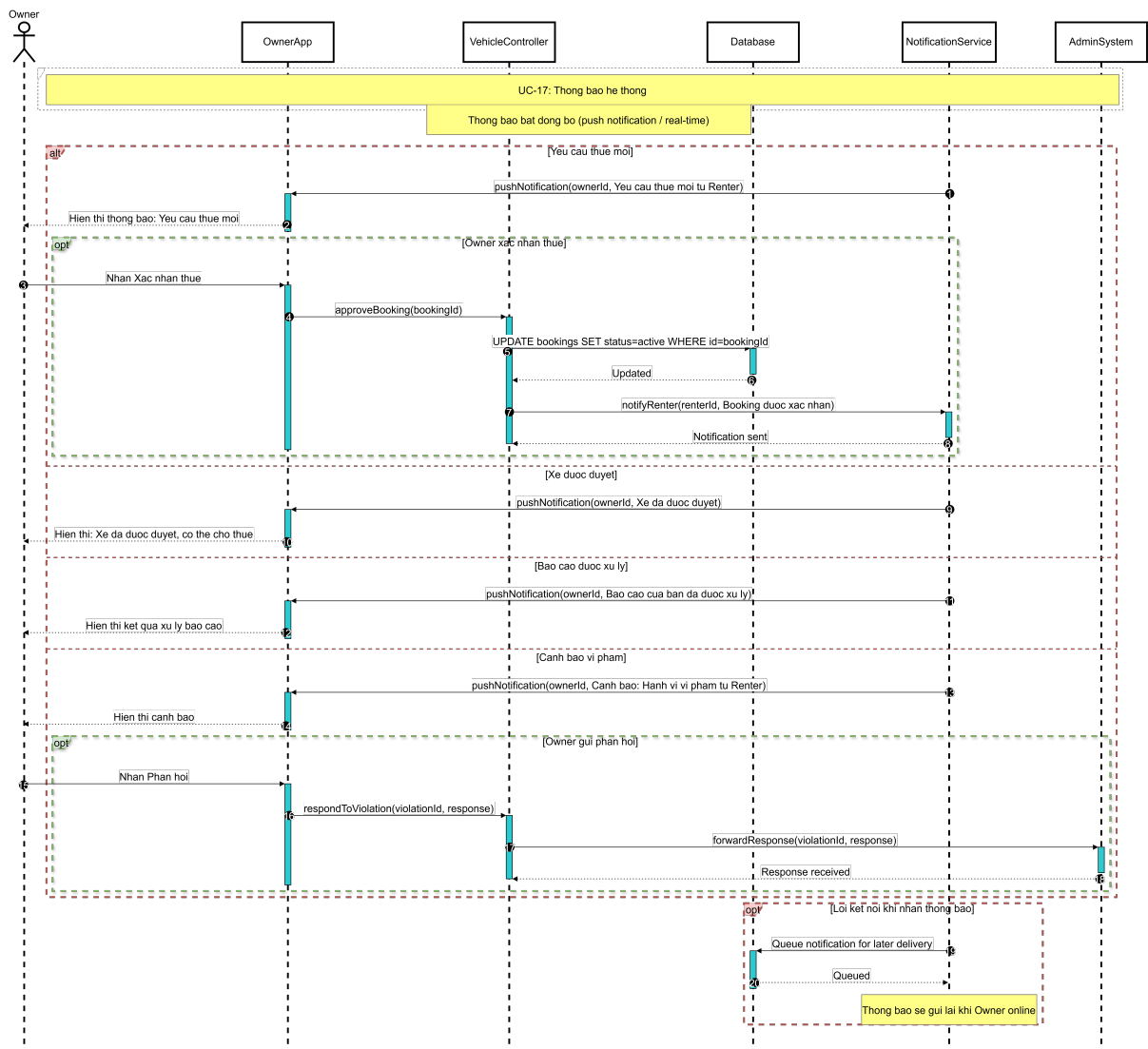


Figure 15: Sequence Diagram - Thông báo hệ thống

Admin Flow

5.3.8 Duyệt xe / Quản trị hệ thống (UC-10) (Hình ??)

Mô tả tổng quát

Admin chịu trách nhiệm duyệt xe mới đăng ký, xử lý xe vi phạm, và quản trị dữ liệu trong hệ thống.

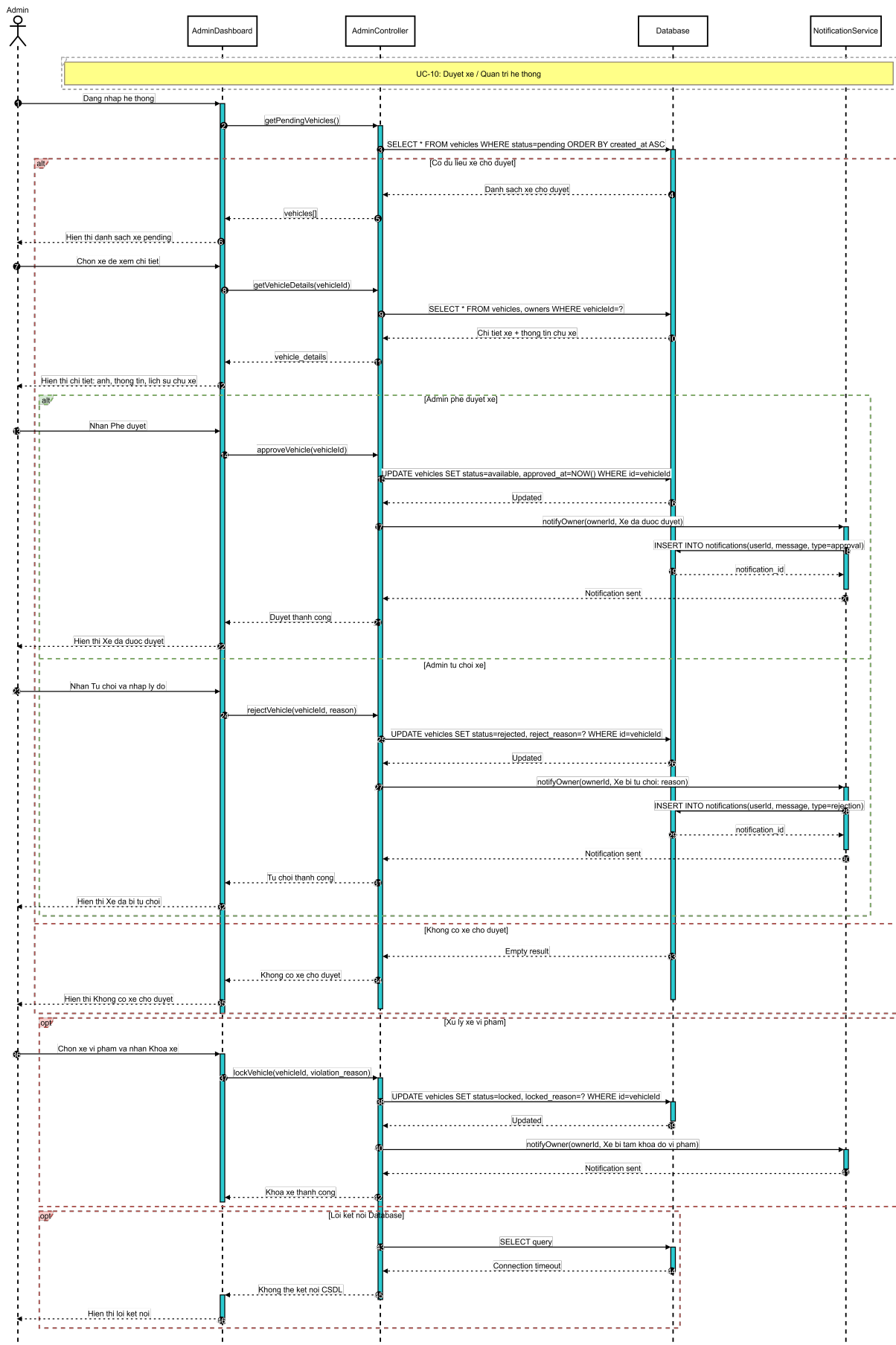


Figure 16: Sequence Diagram - Duyệt xe / Quản trị hệ thống

Thành phần và vai trò

1. **Admin:** Thực hiện duyệt xe, khóa xe, hoặc quản trị chung
2. **AdminDashboard:** Giao diện quản trị hiển thị danh sách xe chờ duyệt
3. **AdminController:** Xử lý logic duyệt xe và thao tác dữ liệu
4. **Database:** Lưu trữ thông tin xe và trạng thái duyệt
5. **NotificationService:** Gửi thông báo cho Owner khi có thay đổi

Hành vi hệ thống

1. Admin đăng nhập vào hệ thống qua AdminDashboard.
2. AdminDashboard gửi yêu cầu đến AdminController để lấy danh sách xe đang chờ duyệt.
3. AdminController truy vấn Database để lấy các bản ghi có status="pending".
4. Hệ thống hiển thị danh sách xe đang chờ duyệt để Admin xem chi tiết từng xe.
5. Khi Admin chọn một xe, AdminDashboard gửi yêu cầu lấy chi tiết xe đến AdminController.
6. AdminController truy vấn Database để lấy thông tin xe và chủ xe tương ứng.
7. Admin xem thông tin chi tiết (loại xe, hình ảnh, pin, vị trí, lịch sử của Owner).
8. Admin chọn "Phê duyệt" hoặc "Từ chối".
9.
 - Nếu phê duyệt: AdminController cập nhật Database status="available" và thông báo cho Owner qua NotificationService.
 - Nếu từ chối: AdminController cập nhật status="rejected" kèm lý do, gửi thông báo từ chối đến Owner.
10. AdminDashboard hiển thị kết quả xử lý cho Admin.
11. Trường hợp có lỗi kết nối Database, hệ thống thông báo lỗi trên giao diện.
12. (Mở rộng) Nếu Admin phát hiện xe vi phạm, Admin có thể chọn "Khóa xe", hệ thống cập nhật status="locked" và gửi thông báo cảnh báo cho Owner.

5.3.9 Xem báo cáo / Thống kê (UC-11) (Hình ??)

Mô tả tổng quát

Admin có thể xem thống kê tổng hợp hệ thống (doanh thu, số lượt thuê, sự cố, tỷ lệ loại xe) theo khoảng thời gian.

Thành phần và vai trò

1. **Admin:** Chọn khoảng thời gian, xuất báo cáo
2. **AdminDashboard:** Giao diện hiển thị biểu đồ thống kê
3. **AnalyticsService:** Tổng hợp và xử lý dữ liệu thống kê
4. **Database:** Cung cấp dữ liệu thô (booking, report, revenue)

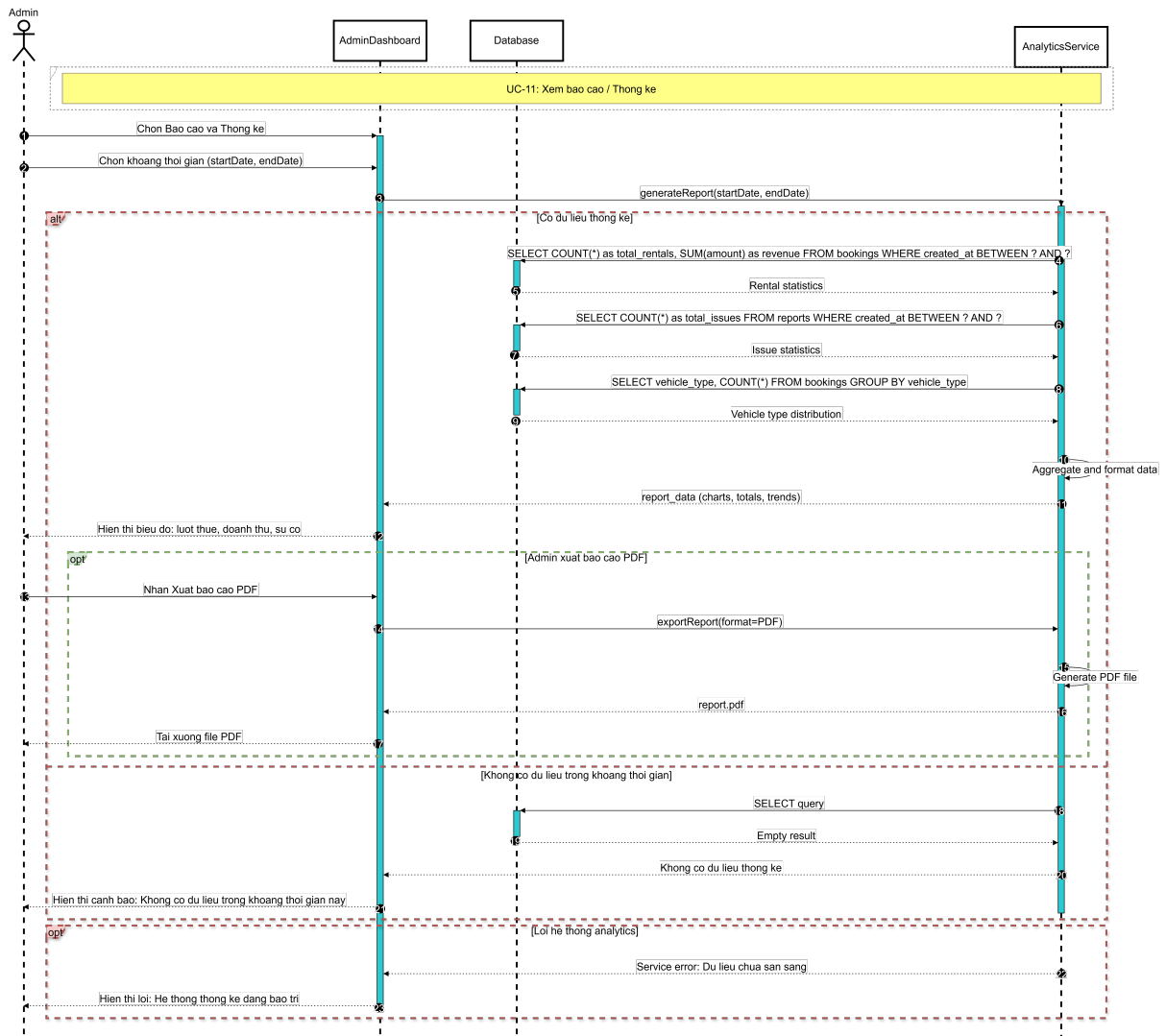


Figure 17: Sequence Diagram - Xem báo cáo / Thống kê

Hành vi hệ thống

- Admin chọn mục "Báo cáo & Thống kê" trên AdminDashboard.
- Admin nhập khoảng thời gian cần thống kê (startDate, endDate).
- AdminDashboard gửi yêu cầu đến AnalyticsService để tạo báo cáo.
- AnalyticsService truy vấn Database để lấy:
 - Tổng số lượt thuê xe, doanh thu.
 - Số lượng sự cố báo cáo.
 - Phân bố loại xe được thuê.
- AnalyticsService tổng hợp và định dạng dữ liệu thành biểu đồ, số liệu thống kê, xu hướng.
- Hệ thống hiển thị báo cáo trực quan cho Admin.
- (Mở rộng) Nếu Admin muốn xuất file, chọn "Xuất báo cáo PDF", AnalyticsService tạo file PDF và gửi về dashboard để tải xuống.
- Nếu không có dữ liệu trong khoảng thời gian chọn, hệ thống hiển thị thông báo "Không có

dữ liệu".

9. Nếu hệ thống phân tích (AnalyticsService) gặp lỗi, hiển thị cảnh báo "Hệ thống thống kê đang bảo trì."

5.3.10 Quản lý người dùng (UC-12)

(Hình ??)

Mô tả tổng quát

Chức năng cho phép Admin theo dõi, khóa, mở khóa tài khoản và xem điểm uy tín (Trust Score) của từng người dùng.

Thành phần và vai trò

1. **Admin:** Giám sát và điều chỉnh trạng thái tài khoản
2. **AdminDashboard:** Hiển thị danh sách và chi tiết người dùng
3. **AdminController:** Xử lý yêu cầu thay đổi trạng thái
4. **Database:** Lưu thông tin người dùng, trạng thái, trust_score
5. **NotificationService:** Gửi thông báo đến người dùng
6. **TrustEngine:** Phân tích và tính toán điểm tin cậy

Hành vi hệ thống

1. Admin chọn mục "Quản lý người dùng" trên AdminDashboard.
2. AdminDashboard gửi yêu cầu đến AdminController để lấy danh sách người dùng.
3. AdminController truy vấn Database và trả về danh sách gồm thông tin: họ tên, email, vai trò (Owner/Renter), trạng thái tài khoản, và điểm tin cậy (Trust Score).
4. AdminDashboard hiển thị danh sách trên màn hình.
5. Admin chọn một người dùng để xem chi tiết.
6. AdminController truy vấn Database lấy thông tin tài khoản, lịch sử thuê xe, đánh giá và vi phạm của người đó.
7. Hệ thống hiển thị chi tiết lịch sử hoạt động của người dùng.
8.
 - Nếu Admin muốn khóa tài khoản, nhập lý do → AdminController cập nhật status="locked" trong Database → gửi thông báo cho user qua NotificationService.
 - Nếu Admin muốn mở khóa tài khoản, cập nhật status="active" và gửi thông báo xác nhận.
9. Admin có thể chọn "Xem Trust Score chi tiết", hệ thống truy vấn TrustEngine để hiển thị điểm tin cậy và các yếu tố ảnh hưởng (đánh giá, hủy đơn, vi phạm).
10. Nếu xảy ra lỗi truy vấn Database, hệ thống hiển thị thông báo lỗi kết nối hoặc không thể truy xuất dữ liệu.

5.3.11 Hệ thống Trust Score (UC-13) (Hình ??)

Mô tả tổng quát

Trust Score Engine là mô-đun AI đánh giá độ uy tín người dùng dựa trên đánh giá, tần suất vi phạm, tỷ lệ hủy đơn, và lịch sử giao dịch.

Thành phần và vai trò

1. **Admin:** Có thể kích hoạt cập nhật thủ công hoặc theo dõi điểm tin cậy
2. **TrustEngine:** Mô-đun AI tính toán điểm tin cậy
3. **Database:** Cung cấp dữ liệu hành vi người dùng
4. **NotificationService:** Gửi cảnh báo cho admin khi điểm thấp
5. **AdminDashboard:** Hiển thị điểm Trust Score và cảnh báo

Hành vi hệ thống

1. Admin có thể kích hoạt cập nhật điểm tin cậy thủ công cho người dùng thông qua AdminDashboard.
2. AdminDashboard gửi yêu cầu đến TrustEngine để thực hiện tính toán lại.
3. TrustEngine truy xuất dữ liệu từ Database, bao gồm:
 - Điểm đánh giá trung bình.
 - Tỷ lệ hủy đơn.
 - Số lần vi phạm được báo cáo.
4. TrustEngine tính toán lại Trust Score dựa trên mô hình AI tích hợp ($\text{trust_score} = f(\text{rating}, \text{cancellation}, \text{violation})$), sau đó cập nhật kết quả trở lại Database.
5. NotificationService gửi thông báo đến AdminDashboard về việc cập nhật thành công.
6. Trong trường hợp thiếu dữ liệu hoặc lỗi mô hình, hệ thống hiển thị "Không đủ dữ liệu để cập nhật Trust Score."
7. Ngoài ra, TrustEngine hoạt động tự động (event-driven) — khi có đánh giá mới hoặc vi phạm, hệ thống tự động cập nhật điểm tin cậy người dùng.
8. TrustEngine định kỳ quét Database để phát hiện người dùng có $\text{trust_score} < 30$ và gửi cảnh báo đến AdminDashboard.
9. Admin nhận cảnh báo và có thể xem chi tiết người dùng bị đánh giá thấp để kiểm tra hoặc xử lý.

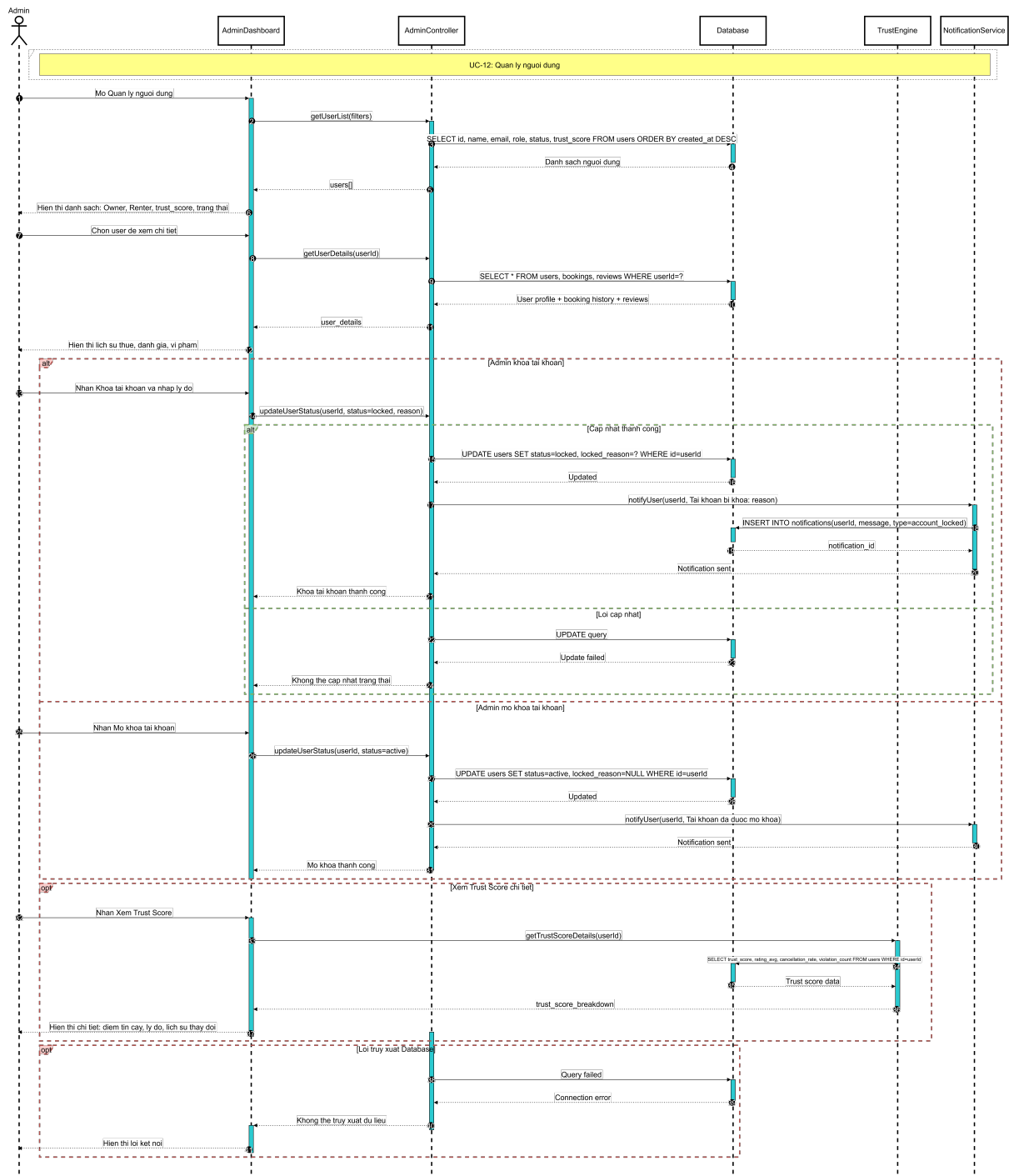


Figure 18: Sequence Diagram - Quản lý người dùng

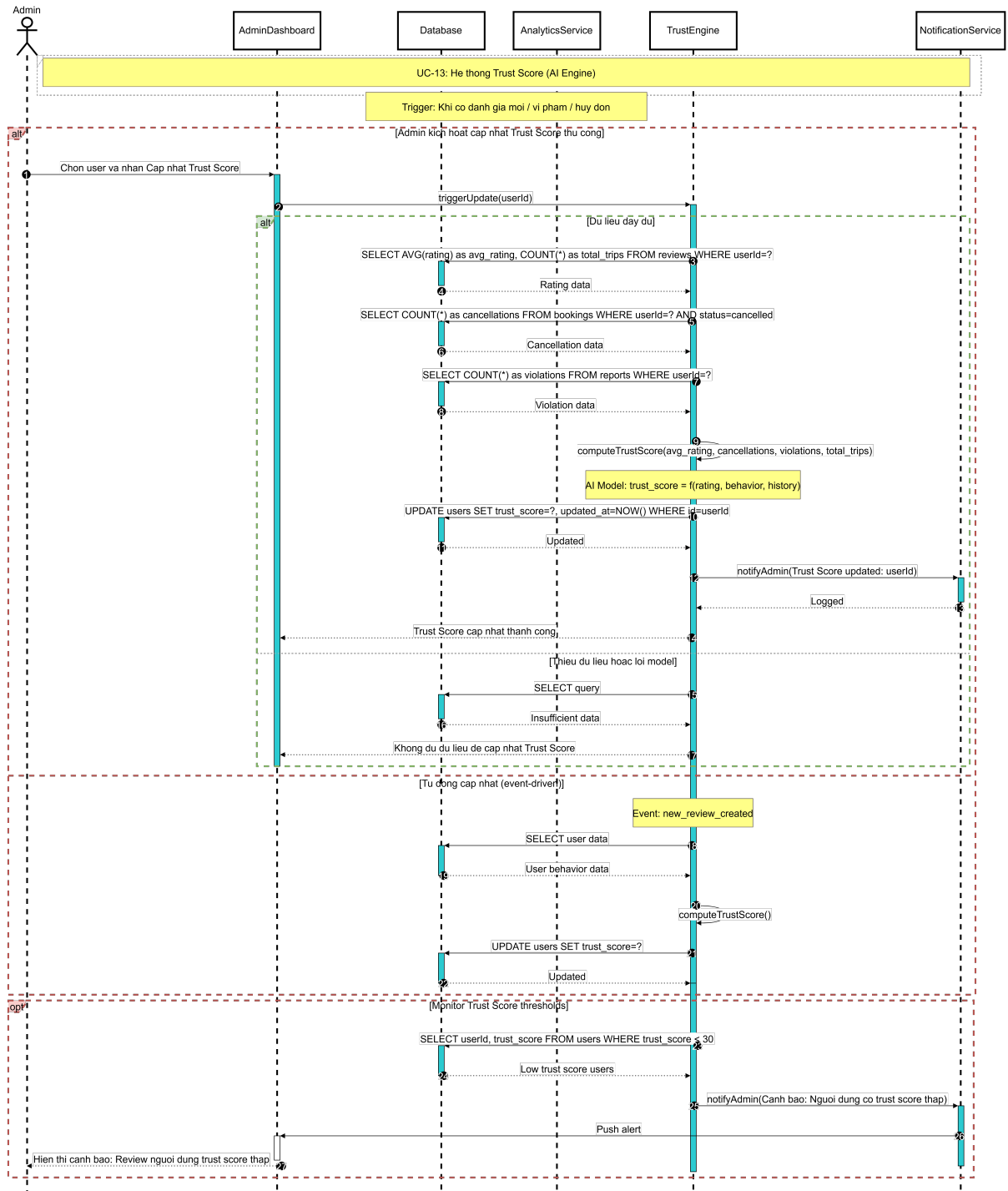


Figure 19: Sequence Diagram - Đánh giá uy tín người dùng

5.4 Logical data flows

6 Thiết kế hệ thống

6.1 Kiến trúc hệ thống

6.1.1 Đặc trưng của hệ thống

Hệ thống có những **đặc điểm kỹ thuật và nghiệp vụ** sau đây, ảnh hưởng trực tiếp đến việc chọn kiến trúc:

Nhóm đặc trưng	Mô tả chi tiết
1. Nhiều loại người dùng	Hệ thống phục vụ 3 vai trò: người thuê, người chia sẻ (chủ xe), và admin. Cả 2 vai trò chính (renter/owner) đều có thể là cùng 1 người dùng.
2. Giao tiếp thời gian thực nhẹ	Ứng dụng cần cập nhật vị trí xe, tình trạng pin, và trạng thái thuê xe theo thời gian gần thực (real-time-ish) nhưng không yêu cầu tốc độ quá cao như game hay trading.
3. Dữ liệu không đồng bộ	Xe có thể được thêm, xóa, cập nhật vị trí, trạng thái thuê... thường xuyên.
4. Hỗ trợ di động và web	Người dùng có thể thao tác qua ứng dụng di động (React Native) hoặc trang web (React.js).
5. Khối lượng truy cập phân tán, có thể tăng dần	Hệ thống hướng đến cộng đồng dân cư / trường học, sau đó có thể mở rộng ra quy mô thành phố. Cần dễ mở rộng (scalable).
6. Có mô-đun AI tích hợp	Module dự đoán thời gian thuê hiệu quả (ML) chạy độc lập hoặc song song với backend chính.
7. Có yếu tố học thuật P2P	Mô phỏng cơ chế chia sẻ ngang hàng: người dùng vừa thuê vừa cho thuê, có cơ chế đánh giá uy tín (trust score).
8. Cần bảo mật và xác thực	Dữ liệu người dùng, vị trí và giao dịch phải được bảo vệ bằng cơ chế xác thực và mã hóa.
9. Tính mở rộng công nghệ	Dễ tích hợp thêm IoT (nếu gắn cảm biến pin xe), thanh toán điện tử, hoặc AI nâng cao sau này.

Phân tích yêu cầu kỹ thuật (non-functional requirements)

Tiêu chí	Mức yêu cầu	Ý nghĩa
Hiệu năng	Trung bình → Cao	Phản hồi trong $\leq 2s$ cho các thao tác chính (tìm xe, thuê xe).
Scalability (mở rộng)	Quan trọng	Có thể mở rộng khi số người thuê tăng nhanh.
Availability (sẵn sàng)	Trung bình	Không yêu cầu 24/7 nhưng phải ổn định khi sử dụng.
Maintainability (dễ bảo trì)	Cao	Kiến trúc module tách biệt, dễ nâng cấp.
Security (bảo mật)	Quan trọng	Xác thực người dùng, mã hóa JWT, ẩn vị trí chính xác của chủ xe.
Interoperability (tích hợp)	Quan trọng	Dễ tích hợp AI module, API bản đồ, thanh toán.

6.1.2 Các Architectural Styles phổ biến

Style	Mô tả	Ưu điểm	Nhược điểm
Layered	Chia thành các tầng: Presentation, Application, Domain, Infrastructure. Mỗi tầng chỉ giao tiếp với tầng liền kề.	Dễ bảo trì và mở rộng. Thay đổi một tầng không ảnh hưởng tầng khác.	Phức tạp khi có nhiều tầng. Dễ rò rỉ logic giữa các tầng.
Monolithic	Toàn bộ ứng dụng (UI, logic, DB) trong một package duy nhất.	Đơn giản, dễ phát triển và deploy. Phù hợp dự án nhỏ/MVP.	Khó scale. Deploy một phần phải restart toàn bộ. Logic chồng chéo.
Microservices	Chia ứng dụng thành các service độc lập, giao tiếp qua API/message queue.	Scale riêng từng service. Chịu lỗi tốt. Hỗ trợ CI/CD.	Hạ tầng phức tạp (Gateway, orchestration). Debug khó. Chi phí cao.

Style	Mô tả	Ưu điểm	Nhược điểm
Event-driven	Các module giao tiếp qua sự kiện (event bus/pub-sub).	Tốt cho real-time và async. Dễ mở rộng module mới.	Thiết kế phức tạp. Cần message broker (Kafka, RabbitMQ).
Microkernel (Plugin)	Core system nhỏ + các plugin mở rộng chức năng.	Dễ thêm module mới. Core gọn, dễ bảo trì.	Quản lý interface phức tạp. Ít framework hỗ trợ.
Serverless / FaaS	Chạy các hàm độc lập trên cloud (AWS Lambda, Firebase Functions).	Không quản lý server. Tự động scale.	Quản lý state khó. Cold start delay. Chi phí cao khi scale lớn.

Tổng kết mức độ phù hợp với hệ thống chia sẻ xe máy điện P2P

Style	Mức độ	Vai trò / Lý do
Layered	Rất cao ★★★★☆	Style chính cho toàn hệ thống. Nhiều module (user, xe, AI, P2P) cần tách tầng rõ ràng để dễ quản lý, bảo trì và mở rộng.
Event-driven	Cao ★★★★☆☆	Cơ chế giao tiếp bổ sung. Phù hợp real-time (vị trí xe, trạng thái thuê), AI training (học từ event), P2P sync, notification.
Microkernel	Trung bình ★★☆☆☆	Ý tưởng plugin (AI, Payment) tốt cho mở rộng module sau này, nhưng không cần triển khai riêng trong đồ án.
Microservices	Trung bình ★★☆☆☆	Ý tưởng tách module (P2P, AI, Payment) phù hợp, nhưng setup phức tạp , quá nặng cho quy mô đồ án. Có thể mô phỏng bằng Clean Architecture.
Monolithic	Thấp ★☆☆☆☆	Không phù hợp. Hệ thống cần mở rộng nhiều module (AI, P2P, payment) nên kiến trúc nguyên khối sẽ cồng kềnh, khó bảo trì.
Serverless	Thấp ★☆☆☆☆	Chỉ phù hợp tác vụ nhỏ lẻ (AI inference, notification). Toàn hệ thống cần quản lý state (rental, payment, peer state) nên không nên dùng serverless.

6.1.3 Layered Architecture

Tuy nhiên trong phạm vi Layered Architecture, có nhiều pattern triển khai như 3-Tier, MVC, hay Clean Architecture. Để đánh giá kiến trúc phù hợp nhất với đề tài, cần có so sánh chi tiết giữa các Architecture pattern này.

a. MVC (Model – View – Controller)

Mô hình MVC được giới thiệu lần đầu bởi Trygve Reenskaug vào những năm 1970, nhằm tách biệt rõ ba phần chính trong ứng dụng: dữ liệu (Model), giao diện (View), và điều khiển (Controller).

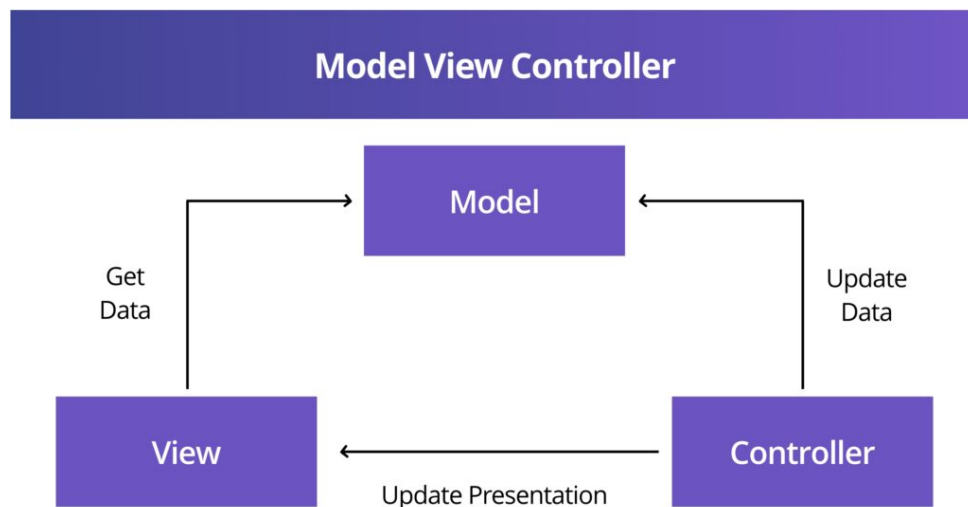


Figure 20: *MVC Architectural Design Pattern*

Thành phần	Vai trò
Model	Quản lý dữ liệu và logic nghiệp vụ của ứng dụng. Model nhận yêu cầu từ Controller, xử lý và trả dữ liệu lại.
View	Chịu trách nhiệm hiển thị dữ liệu của Model ra giao diện, không chứa logic nghiệp vụ.
Controller	Xử lý thao tác từ người dùng (user input), thực hiện xác thực (nếu cần), gọi Model để thay đổi dữ liệu, sau đó cập nhật lại View.

Ưu điểm

- Tách biệt giữa dữ liệu, giao diện và điều khiển → dễ bảo trì.
- Phù hợp với ứng dụng nhỏ, ít tầng logic.

Nhược điểm

- Controller và View phụ thuộc chặt chẽ → thay đổi UI phải sửa Controller.
- Model có thể bị truy cập trực tiếp bởi View, vi phạm nguyên tắc “Separation of Concerns”.
- Khó kiểm thử (unit test) vì Controller gắn liền với giao diện.

b. MVP (Model – View – Presenter)

MVP được phát triển để khắc phục nhược điểm của MVC, bằng cách tách logic xử lý giao diện ra khỏi View, giúp dễ test và bảo trì hơn.

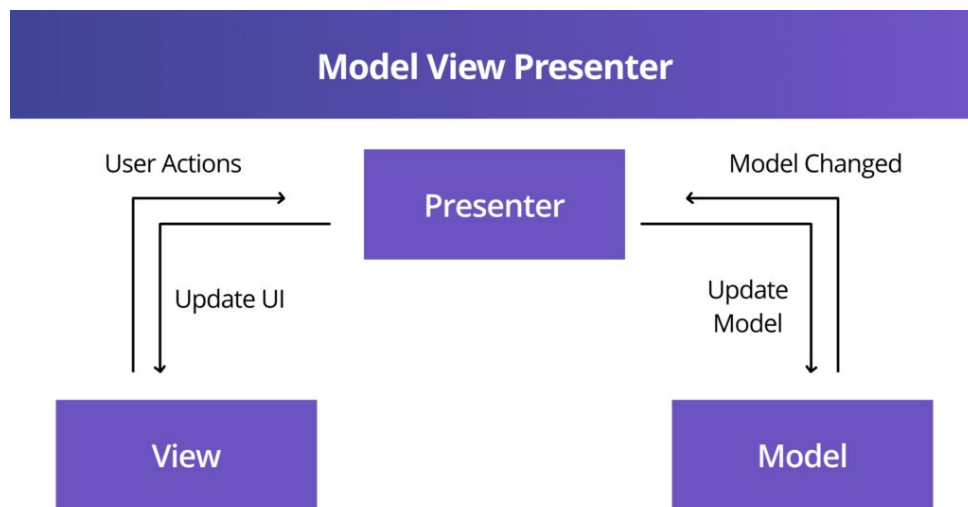


Figure 21: *MVP Architectural Design Pattern*

Thành phần	Vai trò
Model	Tập hợp các lớp mô tả logic nghiệp vụ và dữ liệu. Định nghĩa các quy tắc thao tác với dữ liệu.
View	Hiển thị dữ liệu và chuyển tiếp sự kiện người dùng đến Presenter. Không chứa logic nghiệp vụ.
Presenter	Trung gian giữa View và Model. Nhận input từ View, gọi Model xử lý, sau đó trả kết quả lại cho View để hiển thị.

Ưu điểm

- View không biết Model tồn tại, chỉ nhận dữ liệu từ Presenter → tách biệt rõ ràng.
- Dễ viết unit test cho Presenter vì không phụ thuộc vào UI framework.
- Tăng khả năng tái sử dụng View và Model.

Nhược điểm

- Presenter dễ trở thành “God class” nếu chứa quá nhiều logic.
- Code nhiều hơn do phải tạo các interface giữa View ↔ Presenter.
- Quan hệ Presenter–View vẫn còn chặt (Presenter phải biết View để gọi update).

c. MVVM (Model – View – ViewModel)

MVVM ra đời để loại bỏ sự phụ thuộc giữa Presenter và View trong MVP, bằng cách dùng cơ chế data binding hai chiều. Mô hình này được sử dụng phổ biến trong Android (Jetpack), iOS (SwiftUI) và Frontend (React, Vue, Angular).

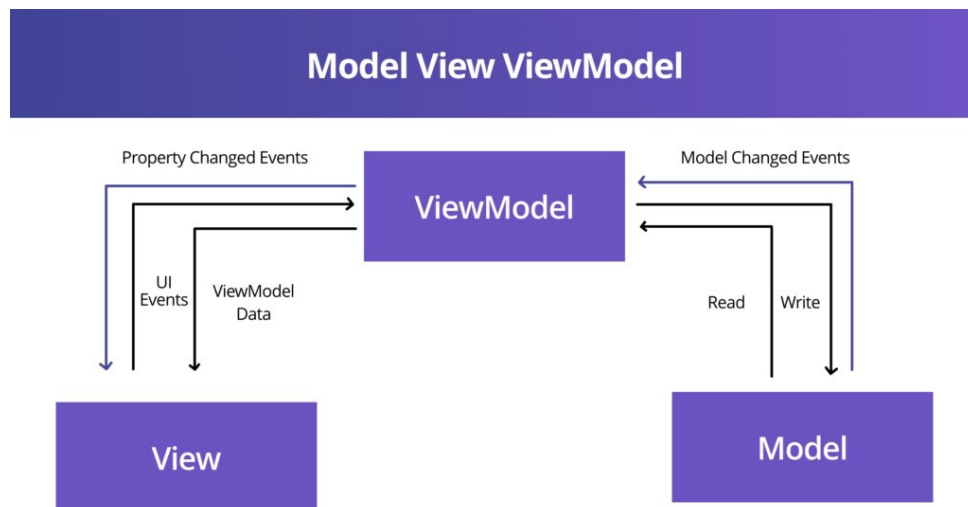


Figure 22: *MVVM Architectural Design Pattern*

Thành phần	Vai trò
Model	Lưu trữ dữ liệu và logic nghiệp vụ. Không giao tiếp trực tiếp với View. Expose dữ liệu qua các Observable/Live-Data/Flow.
View	Giao diện hiển thị dữ liệu, không chứa logic nghiệp vụ. Quan sát (observe) dữ liệu từ ViewModel và tự động cập nhật khi có thay đổi.
ViewModel	Trung gian giữa Model và View. Gọi Model để lấy dữ liệu, xử lý và cung cấp data stream cho View. Sử dụng callback hoặc observable để cập nhật View.

Ưu điểm

- View và ViewModel không biết nhau trực tiếp → giảm coupling.
- Hỗ trợ binding tự động → code gọn, dễ bảo trì.

- Dễ kiểm thử logic trong ViewModel.

Nhược điểm

- Khó debug nếu có nhiều Observable phức tạp.
- Dễ gây vòng lặp dữ liệu nếu binding hai chiều sai cách.
- Yêu cầu hiểu biết về reactive programming.

d. Clean Architecture

Clean Architecture được đề xuất bởi Robert C. Martin (Uncle Bob), nhằm chuẩn hóa cách tách biệt tầng trong ứng dụng – đặc biệt cho các hệ thống lớn, dễ bảo trì, test và mở rộng. Kiến trúc này được chia thành các vòng (layers) đồng tâm, mọi phụ thuộc đều hướng vào trong.

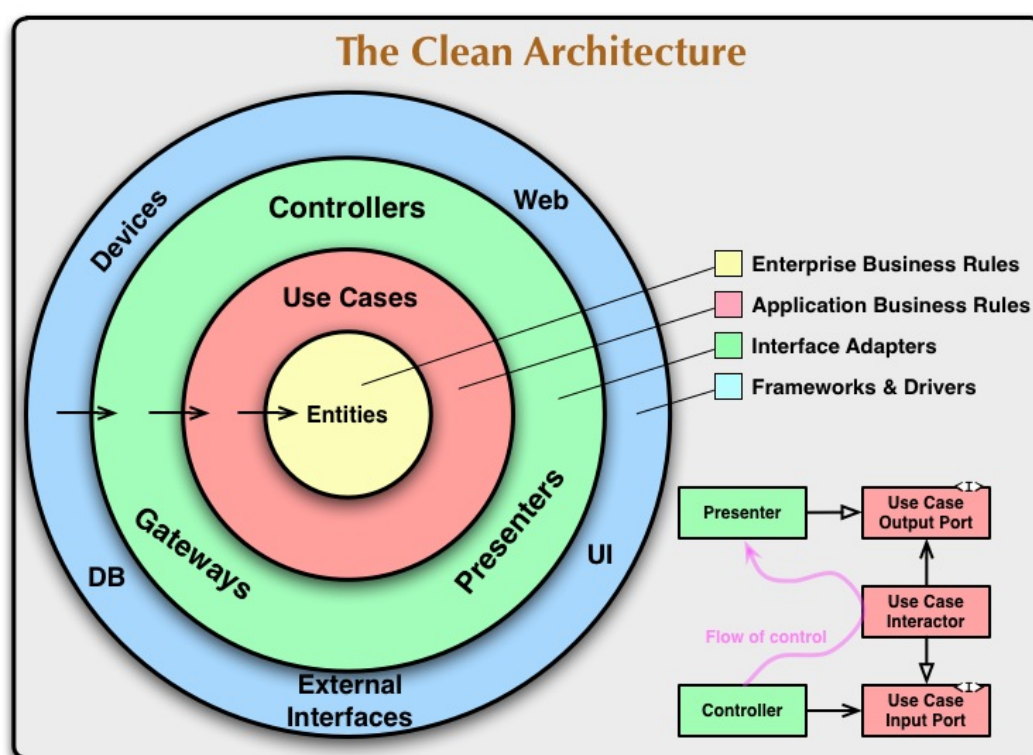


Figure 23: Clean Architecture (Uncle Bob)

Quy tắc phụ thuộc (Dependency Rule): Tất cả phụ thuộc phải chỉ hướng vào trong – nghĩa là lớp ngoài có thể gọi lớp trong, nhưng lớp trong không được biết gì về lớp ngoài.

Các tầng chính

Tầng	Thành phần chính	Vai trò
UI / Interface Layer	Controller, Presenter, ViewModel, View	Tiếp nhận và hiển thị dữ liệu với người dùng.
Application Layer	UseCase, Interactor, InputBoundary, OutputBoundary	Chứa logic nghiệp vụ ứng dụng (use case).
Domain Layer	Entities	Chứa logic nghiệp vụ cốt lõi, không phụ thuộc framework.
Infrastructure Layer	DataAccessInterface, DataAccess, Database	Thao tác lưu trữ dữ liệu (DB, API, file...).

Luồng hoạt động chi tiết

Bước 1 – Controller nhận input từ người dùng

Người dùng gửi request (qua web form, API, hay giao diện). Controller nhận dữ liệu đầu vào (Input Data) và gói lại trong một Data Transfer Object (DTO) – ví dụ `CreateOrderRequest` hoặc `LoginRequest`. Controller không xử lý nghiệp vụ mà chuyển tiếp qua interface `InputBoundary`.

Bước 2 – InputBoundary chuyển dữ liệu vào Use Case

`InputBoundary` là một interface, đảm bảo Controller không phụ thuộc trực tiếp vào logic nghiệp vụ. `UseCaseInteractor` implement `InputBoundary`, nên khi Controller gọi interface này, luồng điều khiển vào bên trong Application Layer.

Bước 3 – Use Case Interactor thực thi nghiệp vụ

Đây là trái tim của ứng dụng (application business rules). `UseCaseInteractor`:

- Gọi Entities để áp dụng logic nghiệp vụ (ví dụ: tính tổng tiền, kiểm tra điều kiện).
- Dùng `DataAccessInterface` để lấy dữ liệu thật từ DB (thông qua repository).
- Kết quả xử lý được đóng gói lại thành `OutputData` (cũng là POJO).

Bước 4 – OutputBoundary trả kết quả ra ngoài

Sau khi xử lý xong, `UseCaseInteractor` gửi `OutputData` qua `OutputBoundary` (interface). `Presenter` implement `OutputBoundary`, nên sẽ nhận được dữ liệu này.

Bước 5 – Presenter chuyển đổi dữ liệu cho UI

`Presenter` không xử lý nghiệp vụ mà chỉ lo định dạng dữ liệu để hiển thị: `ViewModel` là một Data Transfer Object (DTO) chứa dữ liệu UI-friendly.

Bước 6 – View hiển thị dữ liệu cho người dùng

View chỉ cần nhận ViewModel và render ra giao diện (HTML, React component, XML layout, ...). View không chứa logic nghiệp vụ, chỉ hiển thị đúng dữ liệu đã được Presenter chuẩn bị.

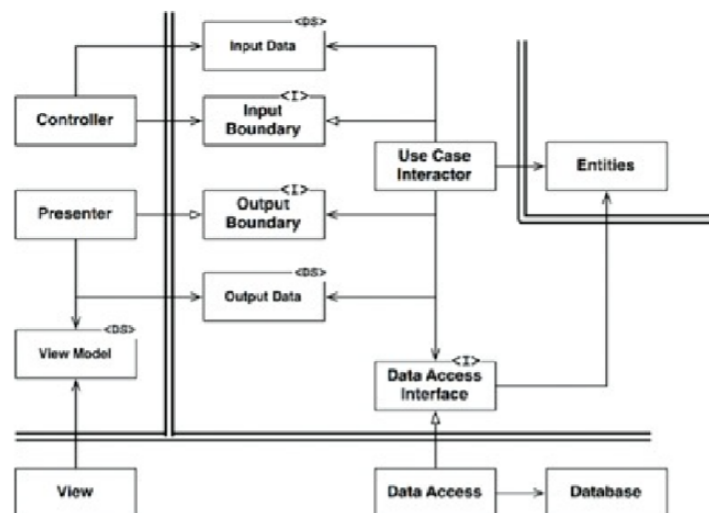


Figure 24: A typical scenario for a system utilizing a database follow Clean Architecture

Ưu điểm

- Mỗi tầng chỉ phụ thuộc vào tầng “bên trong” → thay đổi dễ dàng.
- Dễ test độc lập từng tầng.
- Cho phép thay đổi framework (Android, iOS, Web) mà không động vào logic.

Nhược điểm

- Cấu trúc phức tạp, nhiều lớp.
- Không phù hợp cho dự án nhỏ hoặc nguyên mẫu (MVP app).

Dựa trên những phân tích trên, hệ thống chia sẻ xe máy điện P2P sẽ sử dụng **Clean Architecture** làm kiến trúc chính, kết hợp với **Event-driven Architecture** để xử lý các tác vụ bất đồng bộ và real-time. Cụ thể:

- **Clean Architecture:** Cung cấp cấu trúc tổng thể cho hệ thống, đảm bảo tính module hóa, dễ bảo trì và mở rộng.
- **Event-driven Architecture:** Được tích hợp như một cơ chế giao tiếp bổ sung giữa các module, đặc biệt phục vụ cho việc đồng bộ trạng thái xe, gửi thông báo real-time, và cung cấp dữ liệu cho module AI/ML.
- **Client-Server:** Là nền tảng giao tiếp cơ bản giữa frontend (mobile/web) và backend thông qua RESTful API.

6.2 Lựa chọn công nghệ

6.2.1 Frontend

Mục tiêu

- Cross-platform mobile (Android/iOS)
- Hiển thị bản đồ, GPS, cập nhật real-time
- Kết nối REST API & WebSocket
- Hiệu năng tốt, UI thân thiện

Một số framework front-end phổ biến

Tiêu chí	Flutter (Dart)	React Native (JS/TS)
Hiệu năng	Rất cao (native code)	Tốt (bridge JS-Native)
Tích hợp bản đồ	Google Maps, Mapbox plugin	react-native-maps
Real-time (WebSocket)	socket_io_client	Socket.io (cùng stack)
UI customization	Rất cao, đồng nhất	Phụ thuộc native components
Kích thước APK	Lớn hơn (60-80MB)	Nhỏ hơn (40-50MB)
Cộng đồng	Đang phát triển mạnh	Lớn, nhiều thư viện
Clean Architecture	Tốt nhất (domain-data-presentation)	Khả thi (Redux + layers)
Hot reload	Rất nhanh	Nhanh
Learning curve	Cao (Dart mới)	Thấp (JS phổ biến)
Tích hợp backend Node.js	Tốt (REST/WebSocket)	Rất tốt (cùng JS)
Tái sử dụng code web	Không (riêng Flutter Web)	Có (React.js)

Lựa chọn Flutter:

- **Hiệu năng vượt trội:** Compile ra native code, không cần bridge → mượt mà hơn, quan trọng cho ứng dụng bản đồ real-time

- **UI đồng nhất và đẹp:** Widget system mạnh mẽ, giao diện giống hệt trên iOS và Android
- **Clean Architecture tự nhiên:** Dart hỗ trợ tốt pattern domain-data-presentation, dependency injection rõ ràng
- **Hot reload nhanh:** Tăng tốc độ phát triển, debug dễ dàng
- **Phù hợp dự án học tập:** Thể hiện kỹ năng học công nghệ mới, kiến trúc rõ ràng hơn
- **Google hỗ trợ:** Tài liệu tốt, plugin chính thức (Maps, Location, WebSocket)

Trade-off chấp nhận được:

- Không cùng ngôn ngữ với backend: Vẫn giao tiếp tốt qua REST API và WebSocket, không ảnh hưởng kiến trúc Clean
- APK lớn hơn: Chấp nhận được với mobile hiện đại (storage đủ lớn)
- Learning curve: Dart dễ học, syntax giống Java/JavaScript

6.2.2 Backend

Mục tiêu

- Tương thích Clean Architecture (Layered)
- Hỗ trợ REST API, WebSocket, database
- Tích hợp AI module và P2P service
- Dễ deploy trên cloud
- Dễ mở rộng module (Auth, Payment, AI)

Lựa chọn ngôn ngữ

Ngôn ngữ	Ưu điểm	Nhược điểm / Đánh giá
Java	Chuẩn doanh nghiệp, DI tốt, bảo mật mạnh	Nặng, setup phức tạp. Không phù hợp đồ án.

Ngôn ngữ	Ưu điểm	Nhược điểm / Đánh giá
JavaScript/ TypeScript (Node.js)	Non-blocking I/O, hiệu năng cao. Hỗ trợ real-time tốt (Socket.io). Dễ tổ chức Clean Architecture. Cùng ngôn ngữ với frontend.	Xử lý logic đồng bộ phức tạp cần cẩn thận. Rất phù hợp.
Python	ORM mạnh (Django), dễ tích hợp AI	WebSocket phức tạp (Channels). Cấu trúc MTV cứng nhắc. Phù hợp trung bình.

Lựa chọn: JavaScript/TypeScript (Node.js)

- Cùng ngôn ngữ với frontend React Native → chia sẻ types, dễ debug
- Non-blocking I/O → phù hợp xử lý nhiều request đồng thời
- Hỗ trợ WebSocket tốt → phù hợp P2P và real-time
- Dễ gọi AI service Python qua REST API
- Cộng đồng lớn, dễ deploy

Các framework back-end Node.js phổ biến

Framework	Ưu điểm	Nhược điểm / Đánh giá
Express.js	Nhẹ, linh hoạt, đơn giản	Không có cấu trúc sẵn, cần tự tổ chức. Thiếu DI, decorator. Phù hợp dự án nhỏ.
NestJS	Kiến trúc module rõ ràng. DI, decorator tích hợp sẵn. Hỗ trợ TypeScript tốt. Template Clean Architecture có sẵn. WebSocket built-in.	Phức tạp hơn Express. Learning curve cao hơn. Rất phù hợp Clean Architecture.

Framework	Ưu điểm	Nhược điểm / Đánh giá
Fastify	Hiệu năng cao nhất	Cộng đồng nhỏ hơn. Plugin ít hơn. Phù hợp trung bình.

Lựa chọn: NestJS

- Hỗ trợ Clean Architecture tốt nhất (module, DI, layers)
- Decorator và TypeScript → code rõ ràng, dễ bảo trì
- WebSocket tích hợp sẵn → phù hợp P2P và real-time
- Template có sẵn cho Clean Architecture
- Dễ mở rộng module (Auth, Payment, AI)

Ngoài ra chọn: Python (FastAPI) cho triển khai AI/ML Service sau này

- Python là ngôn ngữ chuẩn cho ML/AI (scikit-learn, TensorFlow, PyTorch)
- FastAPI: hiệu năng cao, async, dễ tích hợp model
- Chạy độc lập như microservice, giao tiếp với backend qua REST API
- Không ảnh hưởng đến kiến trúc chính

6.2.3 Database

Mục tiêu

- Lưu trữ dữ liệu quan hệ (users, vehicles, rentals, payments)
- Hỗ trợ transaction (ACID) cho payment và booking
- Hiệu năng tốt cho truy vấn phức tạp (join, aggregation)
- Khả năng mở rộng và backup dễ dàng
- Tích hợp tốt với NestJS (ORM)

Lựa chọn PostgreSQL:

- **ACID compliance:** Đảm bảo tính toàn vẹn dữ liệu cho payment và rental transactions

- **Spatial data support:** PostGIS extension hỗ trợ location queries (tìm xe gần, tính khoảng cách)
- **JSON support:** Lưu trữ metadata linh hoạt (vehicle specifications, user preferences)
- **TypeORM integration:** NestJS tích hợp sẵn TypeORM, hỗ trợ PostgreSQL tốt nhất
- **Advanced features:** Full-text search, materialized views, trigger cho event-driven
- **Scalability:** Replication, partitioning cho mở rộng sau này
- **Open-source:** Không phí license, cộng đồng lớn

6.3 Sơ đồ triển khai hệ thống

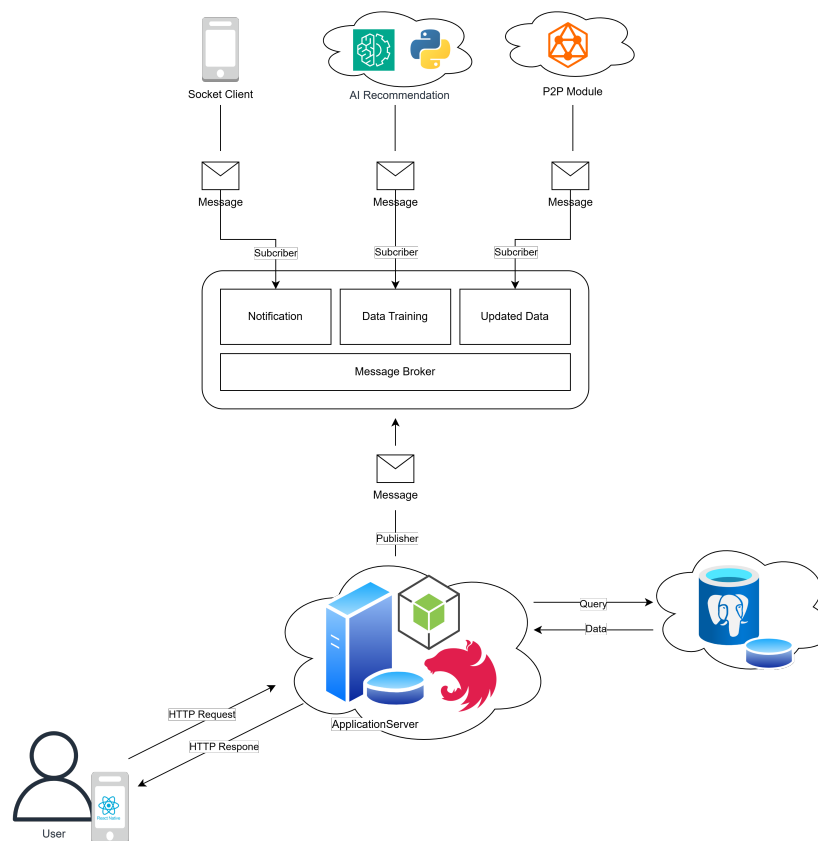


Figure 25: *Deployment Diagram*

7 Thiết kế chi tiết hệ thống

7.1 Database

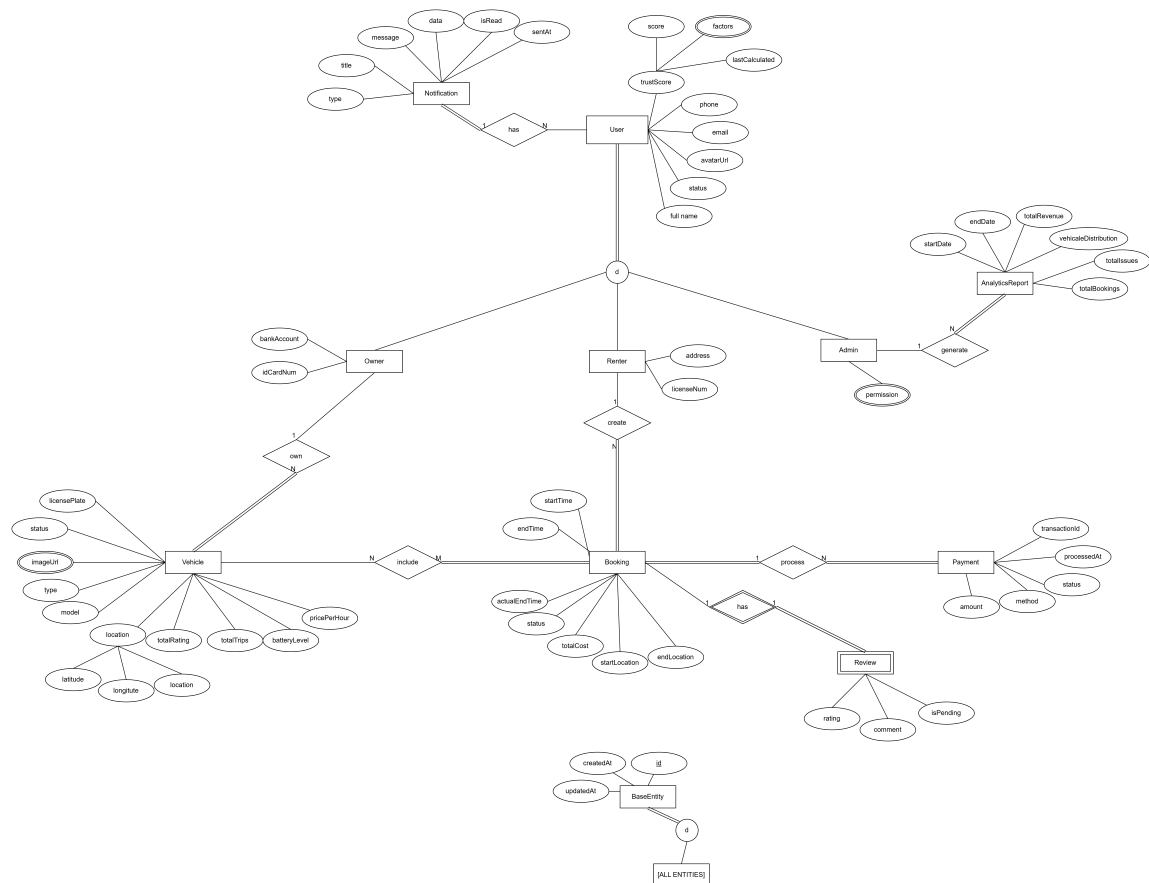


Figure 26: Sơ đồ cơ sở dữ liệu của hệ thống

7.1.1 Các thực thể chính

1. User (cha)

Là thực thể gốc của hệ thống, đại diện cho tất cả người dùng (chủ xe, người thuê, quản trị).

Thuộc tính chính: id, fullName, email, phone, avatarUrl, status, role, createdAt, updatedAt.

Thành phần phức hợp: `trustScore {score, lastCalculated, factors[]}` dùng để đánh giá độ uy tín của người dùng.

2. Renter

Kế thừa từ **User**, biểu diễn người thuê xe.

Thuộc tính riêng: userId (FK), address, licenseNum.

3. Owner

Kế thừa từ **User**, biểu diễn chủ xe.

Thuộc tính riêng: `userId` (FK), `bankAccount`, `idCardNum`.

4. Admin

Cũng kế thừa từ **User**, biểu diễn tài khoản quản trị viên hệ thống.

Thuộc tính riêng: `userId` (FK), `permissions` [] (danh sách quyền thao tác trong hệ thống).

5. Vehicle

Đại diện cho xe máy điện được đăng cho thuê bởi **Owner**.

Thuộc tính chính: `id`, `licensePlate`, `ownerId` (FK), `status`, `type`, `model`, `batteryLevel`, `pricePerHour`, `totalRating`, `totalTrips`, `imageUrl`, `createdAt`.

Thành phần phức hợp: `location` {`latitude`, `longitude`} – vị trí hiện tại của xe dùng cho GPS và bản đồ.

6. Booking

Biểu diễn giao dịch thuê xe giữa **Renter** và **Owner**.

Thuộc tính chính: `id`, `renterId` (FK), `vehicleId` (FK), `startTime`, `endTime`, `actualEndTime`, `status`, `totalCost`, `startLocation`, `endLocation`, `createdAt`.

→ Dùng để quản lý quy trình thuê xe, tính chi phí, và kiểm soát thời gian.

7. Payment

Thực thể lưu thông tin các giao dịch thanh toán liên quan đến **Booking**.

Thuộc tính chính: `id`, `bookingId` (FK), `transactionId`, `amount`, `method`, `status`, `processedAt`.

→ Cho phép quản lý nhiều giao dịch cho cùng một đơn thuê (đặt cọc, thanh toán, hoàn tiền).

8. Review

Đại diện cho đánh giá người thuê dành cho xe hoặc chủ xe sau khi kết thúc **Booking**.

Thuộc tính chính: `id`, `bookingId` (FK), `rating`, `comment`, `isPending`, `createdAt`.

→ Mỗi **Booking** có đúng một **Review** (1:1), giúp tính `totalRating` cho xe.

9. Notification

Dùng để gửi thông báo hệ thống đến người dùng.

Thuộc tính chính: `id`, `userId` (FK), `type`, `title`, `message`, `data` (JSON), `isRead`, `sentAt`.

→ Hỗ trợ gửi thông báo real-time về trạng thái xe, thanh toán hoặc cảnh báo.

10. AnalyticsReport

Được tạo bởi **Admin** nhằm tổng hợp dữ liệu hoạt động của hệ thống.

Thuộc tính chính: id, adminId (FK), startDate, endDate, totalRevenue, totalBookings, totalIssues, vehicleDistribution (JSON), createdAt.

→ Dùng cho mục đích phân tích, báo cáo doanh thu và vận hành.

7.1.2 Quan hệ giữa các thực thể

1. User – Notification (1:N)

Mỗi Notification phải thuộc về đúng một User (tham gia bắt buộc). Một User có thể có không hoặc nhiều thông báo (0..N). Triển khai bằng foreign key **userId** trong bảng Notification với constraint NOT NULL.

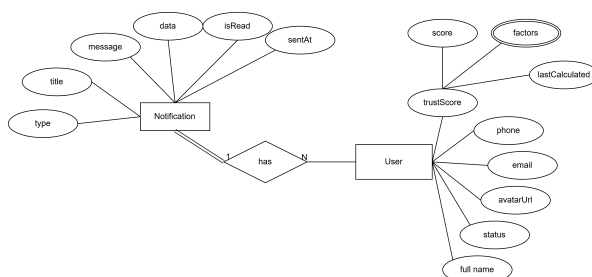


Figure 27: Quan hệ giữa User và Notification (1:N)

2. Admin – AnalyticsReport (1:N)

Mỗi AnalyticsReport được tạo bởi đúng một Admin (tham gia bắt buộc). Một Admin có thể tạo nhiều báo cáo hoặc chưa tạo báo cáo nào (0..N). Foreign key **adminId** trong bảng AnalyticsReport.

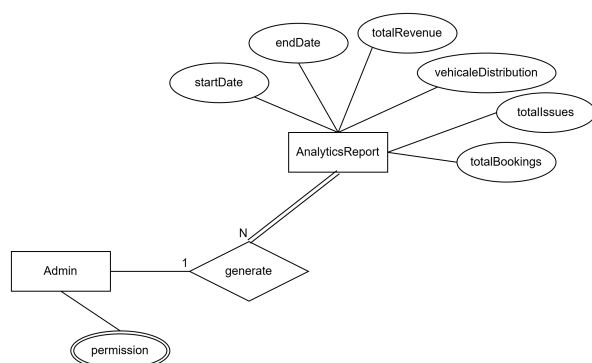


Figure 28: Quan hệ giữa Admin và AnalyticsReport (1:N)

3. Owner – Vehicle (1:N)

Mỗi Vehicle phải thuộc về đúng một Owner (tham gia bắt buộc). Một Owner có thể sở hữu nhiều xe hoặc chưa có xe nào (0..N). Foreign key **ownerId** trong bảng Vehicle với constraint NOT NULL.

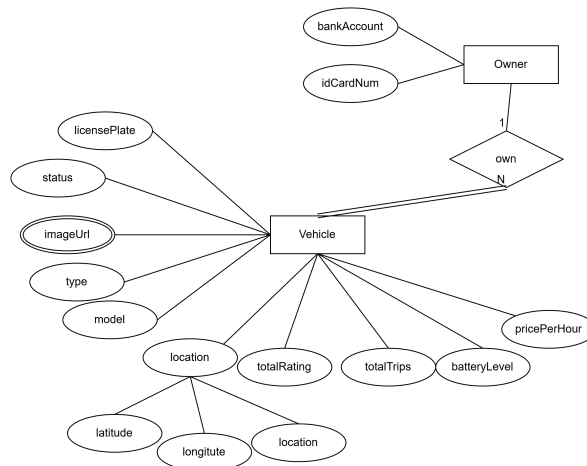


Figure 29: Quan hệ giữa *Owner* và *Vehicle* (1:N)

4. Renter – Booking (1:N)

Mỗi **Booking** được tạo bởi đúng một **Renter** (tham gia bắt buộc). Một **Renter** có thể tạo nhiều booking hoặc chưa có booking nào (0..N). Foreign key **renterId** trong bảng **Booking** với constraint NOT NULL.

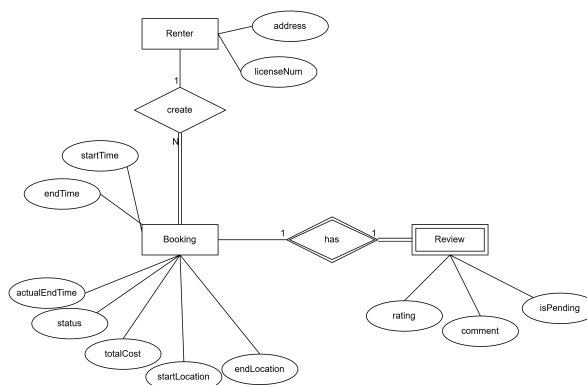


Figure 30: Quan hệ giữa *Renter* và *Booking* (1:N)

5. Booking – Review (1:1)

Mỗi **Review** phải gắn với đúng một **Booking** (tham gia bắt buộc). Mỗi **Booking** có đúng một **Review** (tham gia bắt buộc, quan hệ 1:1). Foreign key **bookingId** trong bảng **Review** với constraint NOT NULL và UNIQUE để đảm bảo mỗi booking chỉ có một đánh giá.

6. Vehicle – Booking (1:N)

Mỗi **Booking** phải gắn với đúng một **Vehicle** (tham gia bắt buộc). Một **Vehicle** có thể có nhiều booking hoặc chưa được thuê (0..N). Foreign key **vehicleId** trong bảng **Booking** với constraint NOT NULL. Hỗ trợ kiểm tra xung đột thời gian thuê xe.

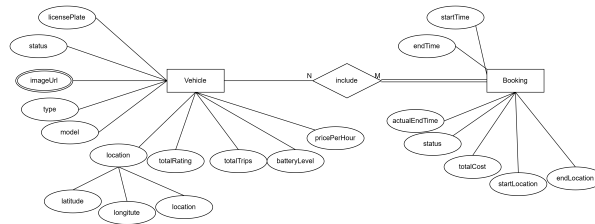


Figure 31: Quan hệ giữa Vehicle và Booking (1:N)

7. Booking – Payment (1:N)

Mỗi Payment phải liên kết đến đúng một Booking (tham gia bắt buộc). Một Booking có thể có nhiều giao dịch thanh toán (0..N) để xử lý các trường hợp: đặt cọc, thanh toán cuối, hoàn tiền, phạt. Foreign key bookingId trong bảng Payment.

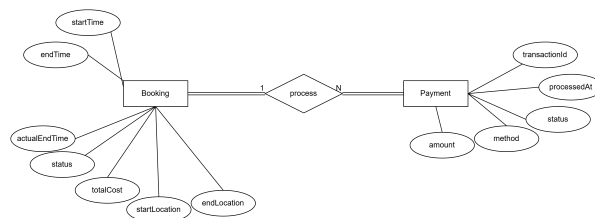


Figure 32: Quan hệ giữa Booking và Payment (1:N)

8. Mối quan hệ kế thừa giữa các thực thể người dùng hệ thống

Mỗi User phải thuộc đúng 1 trong 3 loại: Renter, Owner, hoặc Admin. Renter, Owner, Admin chia sẻ userId (FK → User.id)

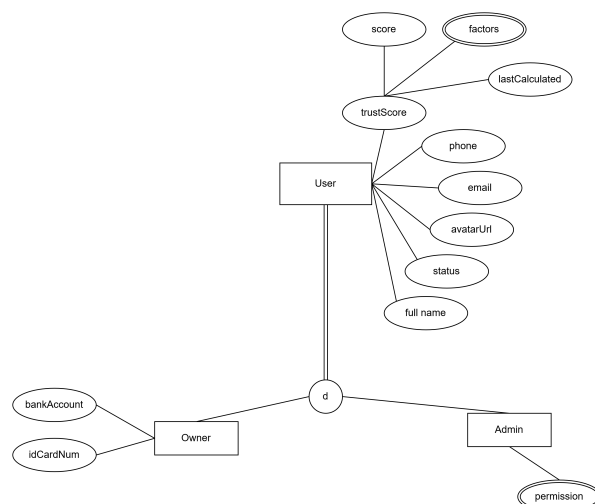


Figure 33: Quan hệ kế thừa giữa User với Renter, User, Admin



8 System Implementation



9 System Evaluation

10 Conclusion

10.1 Achievements

10.2 Limitations

10.3 Further improvements

References

- [1] Flipkart Commerce Cloud, “*The Ultimate Guide to Building a Peer-to-Peer Marketplace*,” Flipkart Commerce Cloud, 2025. Available: <https://www.flipkartcommercecloud.com/guide-to-building-a-peer-to-peer-marketplace/>. (Truy cập ngày 10/10/2025)
- [2] Dân trí, “*Việt Nam có tỷ lệ người sử dụng xe máy cao nhất khu vực Đông Nam Á*,” 14 Tháng 12, 2023. Available: <https://dantri.com.vn/o-to-xe-may/viet-nam-co-ty-le-nguoi-s-u-dung-xe-may-cao-nhat-khu-vuc-dong-nam-a-20231214114542532.htm>
- [3] Kevesko, “*Việt Nam ‘phá đảo’ tỷ lệ người dùng xe máy tại Đông Nam Á*,” 20 Tháng 12, 2023. Available: <https://kevesko.vn/20231220/viet-nam-pha-dao-ty-le-nguoi-dung-xe-may--tai-dong-nam-a-27203888.html>
- [4] PHS, “*Chính sách xe điện ở Việt Nam đã đủ thúc đẩy người dân chuyển đổi*,” 2023. Available: <https://www.phs.vn/tin-tuc/chinh-sach-xe-dien-o-viet-nam-da-du-thuc-day-nguoi-dan-chuyen-doi/6250648>
- [5] VnBusiness, “*Xuất hiện cú hích, thị trường xe máy điện vẫn thiếu hai yếu tố quan trọng*,” 2023. Available: <https://vnbusiness.vn/thi-truong/xuat-hien-cu-hich-thi-truong-xe-may-dien-van-thieu-hai-yeu-to-quan-trong-1108286.html>
- [6] Y. Chen, X. Li, and M. Qiu, “*Buying to share: How prosumption promotes purchases in peer-to-peer asset sharing*,” ResearchGate, 2022. Available: https://www.researchgate.net/publication/358322592_Buying_to_share_How_prosumption_promotes_purchases_in_peer-to-peer_asset_sharing
- [7] O. Ert, A. Fleischer, and N. Magen, “*Trust in the Sharing Economy*,” ResearchGate, 2016. Available: https://www.researchgate.net/publication/299812647_Trust_in_the_Sharing_Economy